

넘파이 연산과 통계

필터링 · 기초 통계 · axis

"경험상 위험하다"를 "데이터상 몇 %가 기준을 넘었다"로 — 진짜 분석의 시작

10_02 데이터 추출과 통계 분석 · PART 02 배열 연산과 필터링 · PART 03 기초 통계

핵심 키워드

요소별 연산 · 스칼라 연산 · 브로드캐스팅 · 정규화 · 불리언 배열

`v[v > 85]` · `np.where()` · `(a) & (b)` · `(a) | (b)` · `.sum()` · `.mean()`

`np.median()` · `.max()` · `.min()` · `.var()` · `.std()` · `axis=0` · `np.loadtxt()` · 3-시그마

2026년 8월 11일 (화) · 16일차 · 39차시

국립금오공과대학교 컴퓨터공학부 박민호

이 책을 읽는 법

뱃지 · 코드블록 · 지난 회차와의 연결

1. 항목 뱃지 — 무엇을 배우는 중인지 한눈에

이번 회차는 **메서드가 압도적으로 많다**. `.sum()`, `.mean()`, `.max()`, `.var()`, `.std()` — 전부 배열 뒤에 점을 찍고 부르는 메서드다. 그런데 딱 하나 `np.median()`만 함수다. 이 예외 하나가 시험과 실습에서 계속 발목을 잡는다. 뱃지 색을 보고 구분하는 습관을 들이자.

뱃지	무엇인가	이번 회차 예
메서드	배열 뒤에 점 + 괄호. 배열에게 시키는 동작	<code>.sum()</code> · <code>.mean()</code> · <code>.var()</code> · <code>.std()</code>
함수	앞에 점이 없음. 모듈 이름이 앞에 온다	<code>np.median()</code> · <code>np.where()</code> · <code>np.loadtxt()</code>
연산자	기호 하나로 동작하는 것	<code>></code> · <code>==</code> · <code>&</code> · <code> </code>
문법	키워드가 아니라 표기 규칙	<code>v[v > 85]</code> · <code>(a) & (b)</code>
자료형	값의 종류 자체	<code>bool</code> 불리언 배열 · <code>float64</code>
개념	문법이 아닌 사고방식 · 배경 지식	벡터 연산 · 브로드캐스팅 · 3-시그마

2. 지난 회차와의 연결 — 15일차가 그대로 이어진다

지난 시간 마지막에 배운 `data[:, 0]`(열 추출)이 오늘의 **출발점**이다. 열 하나를 뽑아내야 조건을 걸고 통계를 낼 수 있다. 오늘은 그 뒤에 붙는 두 단계 — **필터링**과 **통계 요약** — 를 배운다.

15일차에 배운 것	16일차에 이어지는 것	연결 고리
<code>arr * 2</code> 가 진짜 곱셈	<code>a + b</code> · <code>table - base</code>	벡터 연산의 본격 확장
<code>data[:, 0]</code> 열 추출	<code>rpm[rpm > 2000]</code> 조건 추출	열을 뽑아야 조건을 건다
<code>.shape</code> · <code>.dtype</code>	<code>.mean()</code> · <code>.std()</code>	구조 확인 다음이 통계
속성은 괄호 없음	통계는 대부분 메서드 — 괄호 있음	같은 3초 판별법
13일차 파일 입출력	<code>np.loadtxt()</code>	CSV를 곧바로 배열로
12일차 <code>import</code>	보조 교재의 3-시그마 규칙	통계로 이상 정의하기

주의

이번 회차 자료 결론 고지. 수업 전사본(STT)은 녹음 9개·총 5시간 43분 분량인데 **마지막 조각이 실습 5 설명 도중 끊긴다**. 그래서 실습 7·8·9(np.loadtxt 파일 읽기·필터링과 통계 결합·종합 분석)를 강사가 코드로 풀어 준 구간이 전사본에 남아 있지 않다. 이 부분은 강사 배포 코드 10_02_12_practice_39_260811_9.py(정답 + 상세 주석 포함)와 교안 p59~p61로 복원했으므로 내용 자체는 정확하다. 또 병합본 마지막의 numpy_setup_39_260811_4.py는 **내용이 비어 있는 파일**이다.

3. 코드블록 읽는 법 — 초록 화살표는 실행 결과다

이 책의 코드블록에서 **회색 구분선 아래 초록색 ▶**로 시작하는 줄은 전부 실제 실행 결과다. 눈으로 짐작해 적은 것이 아니라 돌려서 받은 출력이다. 결과가 예상과 다르면 그 자리에서 이유를 설명해 두었다.

예시

```
v = np.array([70, 95, 71, 88, 73])
print(v > 85)
```

▶ [False True False True False] ← 이 줄이 실행 결과

4. 오늘 다루는 데이터 — 10_mct_tool.csv

오늘 실습 7·8·9는 전부 이 파일 하나를 쓴다. **CNC 가공(MCT) 설비의 실측 로그** 40행 6열이다. 열 번호는 0부터 세므로, 회전수가 4번·토크가 5번이라는 점을 기억해 두자.

열 번호	열 이름	예시 값	오늘 쓰나
0	공구번호	43000007	아니오
1	사이클타임	21156 · 15611	아니오
2	온도	30 · 29	아니오
3	전류	15.44 · 62.48	아니오
4	회전수 (RPM)	4603 · 4967 · 58	예 — 주력
5	토크	2 · 21 · 147	예 — 주력

설비 적용

회전수 40개 값 중 **4603이 대부분**이고, 중간에 **58**이 한 번, **1216·1217·1241**이 세 번 찍혀 있다. 58은 사실상 공구가 멈춘 순간이다. 오늘 배우는 필터링·통계는 결국 **이 58을 사람 눈이 아니라 코드로 찾아내는 일**이다.

CONTENTS

목차

오늘 배운 것 전체 지도

장 · STEP	무엇을 배우나	핵심
CHAPTER 0	오늘의 지도 — 넘파이 5단계 워크플로우와 용어 분류	불러오기→통계 요약
STEP 1	벡터 연산 — 반복문을 지우는 세 가지 방식	$a+b$ · $a*2$ · 브로드캐스팅
STEP 2	정규화 — 벡터 연산의 첫 실전 (실습 3)	$(x-\min)/(\max-\min)$
STEP 3	비교 연산과 불리언 배열 — 조건을 배열로	$v > 85$ · <code>.sum()</code>
STEP 4	값 골라내기 — Boolean indexing과 <code>np.where</code>	<code>v[v>85]</code> · <code>np.where()</code>
STEP 5	다중 조건 — 괄호와 <code>&</code> (최다 실수 구간)	$(a) \& (b)$
STEP 6	기초 통계 ① 중심과 범위 — 평균의 함정	<code>mean</code> · <code>median</code> · <code>max</code>
STEP 7	기초 통계 ② 흩어짐 — 정상의 정의를 세우기	<code>var</code> · <code>std</code> · 3σ
STEP 8	axis — 통계를 어느 방향으로 낼 것인가	<code>axis=0</code> 센서별
STEP 9	파일에서 배열로 — <code>np.loadtxt</code> 와 종합 분석	<code>np.loadtxt()</code>
부록 A	치트시트 — 오늘 나온 연산·통계 전체 요약	한 장 요약
부록 B	오류·함정 사전 + 제출 코드 교정 · 교안 오탈자	$\times \rightarrow \checkmark$
부록 C	실습 3~9 하이라이트 · 보충답안 · 다음 시간 예고	Pandas 예고

오늘로 완성되는 다섯 단계

교안 10_02의 마지막 장(p62)은 넘파이 분석의 전체 흐름을 다섯 칸으로 정리한다. **지난 시간에 1~3단계를 배웠고, 오늘 4·5단계를 채우면서 흐름이 완성된다.** 날개 함수를 외우려 하지 말고 이 다섯 칸 중 어디에 쓰는 도구인지로 묶어 기억하자.



설비 적용

오늘 쓰는 데이터는 **CNC 가공(MCT)** 공정의 실측 로그 `Data/10_mct_tool.csv`다. 40행 × 6열(공구번호 · 사이클타임 · 온도 · 전류 · 회전수 · 토크)이고, 그중 **회전수(4번 열)와 토크(5번 열)**를 주로 쓴다. 실제 값을 보면 회전수 4603이 대부분인데 중간에 **58**이 한 번, 1216 · 1217 · 1241이 세 번 찍혀 있다. 공구가 멈췄거나 이상 정지가 난 시점이다. 오늘 배우는 필터링 · 통계는 결국 **이 58을 코드로 찾아내는 일**이다.

이번 회차의 무게중심 — 문법이 아니라 판단

오늘 새로 배우는 문법은 몇 개 안 된다. `&`, `|`, `axis`, `np.where`, `np.loadtxt` 정도다. 나머지는 이미 아는 연산자(`>`, `+`, `-`)와 메서드다. 그런데도 내용이 두꺼운 이유는 **"무엇을 이상으로 볼 것인가"**라는 판단이 계속 끼어들기 때문이다.

질문	기술적으로는	판단이 필요한 지점
어떤 조건을 걸 것인가	<code>rpm > 2000</code> 한 줄	왜 2000인가? 근거는 무엇인가
<code>&</code> 인가 <code> </code> 인가	<code>&</code> 또는 <code> </code> 하나 선택	둘 다여야 이상인가, 하나만으로 이상인가
평균인가 중앙값인가	<code>mean()</code> vs <code>median()</code>	이상치를 반영할 것인가 배제할 것인가
2σ인가 3σ인가	<code>k=2</code> vs <code>k=3</code>	놓치는 걸 줄일까, 오탐을 줄일까
골라낼까 표시할까	<code>v[조건]</code> vs <code>np.where</code>	시점 정보가 필요한가

개념

강사가 필터링을 설명하며 한 말이 정확하다 — **"기술적인 의미로는 비교 연산 · 다중 조건 비교일 뿐이다. 거기에 '이건 조건상 이상한 상황이야, 걸러내야 돼'라고 논리적 판단을 해서 의미를 부여하니까 필터링이라고 부르는 것."** 코드는 도구고, 기준을 정하는 건 사람이다.

오늘의 지도

왜 통계인가 · 메서드와 함수를 가르는 예외 하나

0-1. 통계가 왜 필요한가 — 정상을 알아야 이상을 안다

설비 로그가 하루 수백만 줄 쌓인다. 그걸 눈으로 다 볼 수는 없다. 그래서 **몇 개의 대표 숫자로 압축**한다. 평균이 얼마인지, 값들이 얼마나 흩어졌는지만 알아도 데이터의 성격이 거의 드러난다.

그런데 통계의 진짜 목적은 요약이 아니다. **"정상"을 숫자로 정의하기 위해서**다. 정상 범위를 세워 두어야 그 밖으로 벗어난 값을 "이상"이라고 부를 수 있다. 강사가 반복해서 말한 문장이 이것이다 — **"정상을 알아야 이상을 안다."**

통계	알려주는 것	설비 분석에서의 의미
평균 mean	대표적인 중심값	이 설비의 평상시 상태
중앙값 median	줄 세웠을 때 한가운데	이상치에 안 흔들리는 중심
최대·최소 max min	값이 퍼진 폭	위험했던 최고 순간
분산 var	평균에서 흩어진 정도(제곱 단위)	요동침의 크기
표준편차 std	흩어진 정도(원래 단위)	정상 범위의 폭 — 이상 탐지 기준

개념

오늘 배우는 통계는 **어려운 수학이 아니라 실용 도구**다. 분산 공식을 손으로 풀 일은 없다. 넘파이가 `.var()` 하나로 계산해 준다. 중요한 건 **어떤 상황에 어떤 통계를 골라 쓸지**, 그리고 **그 숫자를 어떻게 해석할지**다.

0-2. 오늘의 함정 — np.median만 왜 함수인가

오늘 나오는 통계 도구는 거의 다 **메서드**다. 배열 뒤에 점을 찍고 부른다. 그런데 **중앙값만** `np.median(배열)` 형태의 함수다. 교안도 p44에서 **"다른 통계와 달리 np.median 형태"**라고 따로 짚는다.

딱 하나만 다른 형태

```
import numpy as np
s = np.array([70, 72, 71, 95, 73])
```

메서드 — 배열 뒤에 점

```
print(s.sum(), s.mean(), s.max(), s.min())
print(round(s.var(), 2), round(s.std(), 2))
```

함수 — 배열을 인자로 넣는다

```
print(np.median(s))
```

s.median() 은 존재하지 않는다

AttributeError: numpy.ndarray object has no attribute median

```
▶ 381 76.2 95 70
▶ 89.36 9.45
▶ 72.0
```

형태	예	왜 이렇게 갈리나
메서드 배열.함수()	s.sum() · s.mean() · s.std()	ndarray가 직접 갖고 있는 기능
함수 np.함수(배열)	np.median(s) · np.where() · np.loadtxt()	넘파이 모듈이 갖고 있는 기능
둘 다 되는 것	s.mean() = np.mean(s)	대부분의 통계는 양쪽 다 제공

주의

np.mean(s)처럼 함수 형태로도 쓸 수 있는 것이 많아서 더 헷갈린다. 외우는 방법은 하나 — "중앙값만 점을 못 찍는다." s.median()은 AttributeError다. 헷갈리면 전부 np.함수(배열) 형태로 통일해 써도 된다. 그러면 예외를 기억할 필요가 없다.

0-3. 오늘 나온 이름 전체 — 분류부터 해 두고 시작

분류	이름	한 줄
연산자	+ - * /	배열끼리 같은 위치끼리 계산 (요소별 연산)
연산자	> < >= <= == !=	비교 — 결과가 불리언 배열
연산자	& 그리고 · 또는	조건 결합 — 각 조건을 괄호로 묶어야 함
개념	브로드캐스팅	작은 배열이 큰 배열 모양에 맞춰 확장
개념	정규화 (normalization)	값을 0~1 범위로 맞추기

분류	이름	한 줄
자료형	bool 불리언 배열	True/False로 위치를 표시한 배열
문법	v[조건]	Boolean indexing — 조건에 맞는 값만
함수	np.where(조건, 참, 거짓)	조건에 따라 값을 둘 중 하나로
메서드	.sum()	합계 — 불리언 배열이면 True 개수
메서드	.mean()	평균 — 불리언 배열이면 True 비율
함수	np.median()	중앙값 — 이것만 함수
메서드	.max() · .min()	최대·최소
메서드	.var()	분산 — 제곱 단위
메서드	.std()	표준편차 — 원래 단위
키워드	axis=0 · axis=1	통계 방향 — 열별 / 행별
함수	np.round(배열, 자리)	배열 전체 반올림
함수	np.loadtxt()	CSV를 곧바로 배열로 읽기
개념	3-시그마 규칙	평균 $\pm 3 \times$ 표준편차 밖이면 이상

STEP 1

벡터 연산

반복문을 지우는 세 가지 방식

15일차에 `arr * 2`가 진짜 곱셈이 되는 장면을 봤다. 오늘은 그것을 **세 가지 형태로 확장**한다. 셋의 공통점은 하나 — **반복문이 없다**. 이것이 데이터 분석에서 넘파일을 쓰는 가장 큰 이유다.

요소별 연산

배열 + 배열

스칼라 연산

배열 + 숫자

브로드캐스팅

큰 것 + 작은 것

1-1. 요소별 연산 — 배열 + 배열

연산자 `a + b` (요소별 연산) 두 배열을 같은 위치끼리 한 번에 계산

정의 같은 위치의 값끼리 계산한다. 첫째는 첫째끼리, 둘째는 둘째끼리. 더하기·빼기·곱하기·나누기 모두 같은 방식.

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([10, 20, 30])

print(x + y)
print(x - y)
print(x * y)
print(y / x)
```

```
▶ [11 22 33]
▶ [ -9 -18 -27]
▶ [10 40 90]
▶ [10. 10. 10.]
```

왜 쓰나 어제와 오늘 데이터를 빼서 변화량을 구하는 것이 대표 용도다. 센서 10개 × 시점 1000개라도 한 줄로 끝난다.

응용 실무에서 가장 많이 쓰는 형태는 **실측값 - 기준값**이다. 두 배열의 차이를 구해 놓으면 곧바로 편차 분석으로 넘어갈 수 있다. 절댓값을 씌우면 방향과 무관한 오차 크기가 된다.

```
measured = np.array([70.5, 72.1, 95.3, 69.8]) # 실측 온도
target    = np.array([70.0, 70.0, 70.0, 70.0]) # 목표 온도
```

```
diff = measured - target
print("편차 :", diff.round(1))
print("절대 편차:", np.abs(diff).round(1))
print("최대 이탈:", np.abs(diff).max().round(1))
```

```
▶ 편차 : [ 0.5  2.1 25.3 -0.2]
▶ 절대 편차: [ 0.5  2.1 25.3  0.2]
▶ 최대 이탈: 25.3
```

자주 하는 실수

두 배열의 크기가 같아야 한다. 값 3개인 배열과 5개인 배열은 위치를 맞출 수 없어 `ValueError: operands could not be broadcast together with shapes (3,) (5,)`가 난다.

1-2. 스칼라 연산 — 배열 + 숫자

연산자 `a * 2` (스칼라 연산) 배열 전체에 숫자 하나를 한꺼번에 적용

정의 배열의 모든 값에 같은 숫자를 적용한다. 배열 크기와 무관하게 동작한다.

```
celsius = np.array([20.0, 25.0, 30.0])
```

```
# 화씨 = 섭씨 × 1.8 + 32
print(celsius * 1.8 + 32)
```

```
# 크기와 무관 — 백만 개도 한 줄
big = np.arange(1_000_000)
print((big * 2)[:5])
```

```
▶ [68. 77. 86.]
▶ [0 2 4 6 8]
```

왜 쓰나 단위 변환과 센서 오차 보정에 매일 쓰인다. 값이 5개든 백만 개든 코드가 똑같다는 것이 핵심이다.

응용 연산 순서는 **수학과 동일**하다. 곱하기가 먼저, 더하기가 나중이다. 더하기를 먼저 하려면 괄호로 묶는다. 15일차 실습 1에서 화씨 변환을 `* 9/5 + 3`으로 잘못 쓴 적이 있는데, 이런 상수 오류는 **0도와 100도로 계산**하면 즉시 잡힌다.

```
# 검산 습관 — 알려진 값을 넣어 본다
check = np.array([0.0, 100.0])
print(check * 1.8 + 32)      # 0도→32, 100도→212 여야 정상

# 괄호로 순서 바꾸기
a = np.array([1, 2, 3])
print(a * 2 + 10)           # 곱하기 먼저
print(a * (2 + 10))         # 괄호 안이 먼저
```

```
▶ [ 32. 212.]
▶ [12 14 16]
▶ [12 24 36]
```

개념

데이터 크기를 일괄 조정하는 **스케일링**도 스칼라 연산의 한 사례다. 다음 절의 정규화가 바로 그것이다.

1-3. 브로드캐스팅 — 큰 것 + 작은 것

개념 브로드캐스팅 (broadcasting) 작은 배열이 큰 배열 모양에 맞춰 확장

정의 크기가 다른 배열끼리 계산할 때, 작은 쪽을 **도장 찍듯 반복 복사**해 큰 쪽 모양에 맞춘 뒤 계산한다.

```
table = np.array([
    [72, 2.3],
    [95, 6.8]
])

base = np.array([70, 2.0])  # 기준값 한 줄

# 각 행에서 기준값 빼기
print(table - base)
```

```
▶ [[ 2.   0.3]
   [25.   4.8]]
```

왜 쓰나 각 센서가 **기준에서 얼마나 벗어났는지**를 모든 시점에 대해 한 번에 계산할 수 있다. 반복문으로 행마다 돌 필요가 없다.

응용 브로드캐스팅이 성립하려면 **작은 쪽 크기가 1이거나 큰 쪽과 같아야** 한다. 스칼라 연산(숫자 하나)도 사실 브로드캐스팅의 극단적인 경우다 — 숫자 하나가 모든 자리로 퍼지는 것이다. 아래는 40행짜리 실측 데이터에 기준선 한 줄을 적용하는 실전 형태다.

```
# 시점 4개 x 센서 2개 (회전수, 토크)
data = np.array([
    [4603, 2.0], [4967, 21.0], [58, 5.0], [4603, 3.0]
])
```

```
# 센서별 기준값 한 줄
base = np.array([4600, 3.0])
```

```
print("기준 대비 편차:")
print(data - base)
```

▶ 기준 대비 편차:

```
▶ [[ 3.    -1. ]
   [ 367.   18. ]
   [-4542.    2. ]
   [ 3.     0. ]]
```

주의

[-4542.]가 눈에 띈다. 회전수가 기준보다 4542나 낮은 시점 — 공구가 사실상 멈춘 순간이다. 브로드캐스팅 한 줄이 이상 시점을 눈에 보이게 만든다.

1-4. 벡터 연산 정리

종류	대상	예시	크기 조건
요소별 연산	배열 + 배열	$a + b$	두 배열 크기가 같아야 함
스칼라 연산	배열 + 숫자	$a * 2$	크기 무관
브로드캐스팅	큰 배열 + 작은 배열	<code>table - base</code>	작은 쪽이 1이거나 같아야 함

개념

세 가지의 공통점은 **반복문이 필요 없다**는 것이다. 사고방식을 이렇게 옮기자 — "**모든 온도를 화씨로**" 를 for 문으로 옮기지 말고, "**온도 배열에 1.8 곱하고 32 더하기**" 로 바로 옮긴다. 개별 값이 아니라 배열 전체를 대상으로 생각하는 것, 이것이 벡터 연산의 사고방식이다.

주의

큰 데이터로 바로 가지 말고 **작은 배열로 먼저 계산하는 습관**을 권한다. 값 3~4개짜리로 결과를 눈으로 확인한 뒤 실제 데이터에 적용하면, 공식 오류나 축 실수를 훨씬 빨리 잡는다.

STEP 2

정규화

벡터 연산의 첫 실전 — 실습 3

벡터 연산을 배웠으니 곧바로 써 볼 차례다. **정규화(normalization)**는 서로 단위가 다른 값들을 **0과 1 사이로 맞추는** 작업이다. 회전수는 1000~5000, 토크는 1~150 — 이대로는 비교가 안 된다. 같은 자로 재려면 눈금을 맞춰야 한다.

2-1. 정규화 공식과 브로드캐스팅

개념 Min-Max 정규화 값을 0~1 범위로 맞추기

정의 가장 작은 값이 0, 가장 큰 값이 1이 되도록 전체를 비례 축소한다.

형태 정규화된 $x = (x - \text{최솟값}) / (\text{최댓값} - \text{최솟값})$

```
import numpy as np

rpm = np.array([1551, 1408, 1498, 1433, 1425, 2861])

print(rpm.min())
print(rpm.max())

rpm_min = rpm.min()
rpm_max = rpm.max()
normalized = (rpm - rpm_min) / (rpm_max - rpm_min)
print(normalized)
```

```
▶ 1408
▶ 2861
▶ [0.09841707 0.          0.06194081 0.01720578 0.01169993 1.          ]
```

왜 쓰나 단위가 다른 센서를 같은 눈금으로 놓고 비교할 수 있다. 머신러닝 모델에 데이터를 넣기 전 거의 반드시 거치는 전처리이기도 하다.

응용 공식이 전부 벡터 연산이다. $\text{rpm} - \text{rpm_min}$ 은 스칼라 뺄셈(브로드캐스팅), 나누기도 스칼라 나눗셈이다.

반복문이 한 줄도 없다. 결과를 보면 가장 작은 1408이 정확히 0, 가장 큰 2861이 정확히 1이 되었고, 나머지는 그 사이에 촘촘히 몰려 있다 — 2861이 얼마나 튀는 값인지 한눈에 보인다.

```
# 회전수와 토크를 같은 눈금으로
rpm = np.array([1551, 1408, 1498, 2861])
torque = np.array([42.8, 46.3, 49.4, 4.6])

def normalize(a):
    return (a - a.min()) / (a.max() - a.min())

print("회전수 정규화:", np.round(normalize(rpm), 2))
print("토크 정규화:", np.round(normalize(torque), 2))
```

```
▶ 회전수 정규화: [0.1  0.    0.06 1.   ]
▶ 토크 정규화: [0.85 0.93 1.    0.   ]
```

개념

회전수는 마지막이 1(최대), 토크는 마지막이 0(최소)이다. **회전은 가장 빠르는데 힘은 가장 약한 시점** — 공구 이상을 강하게 의심할 수 있다. 정규화는 이런 대칭 관계를 눈에 보이게 만든다.

2-2. np.round — 소수점 자리 정리

함수 `np.round()` 배열 전체를 지정한 자리까지 반올림

정의 배열의 모든 값을 소수점 아래 지정한 자리까지 반올림한 새 배열을 돌려준다.

형태 `np.round(배열, 자릿수)`

```
normalized = np.array([0.09841707, 0.0, 0.06194081, 0.01720578, 0.01169993, 1.0])

print(np.round(normalized, 2))

# 배열 메서드로도 가능
print(normalized.round(2))

# 값 하나는 파이썬 내장 round
print(round(normalized.mean(), 2))

▶ [0.1  0.    0.06 0.02 0.01 1.   ]
▶ [0.1  0.    0.06 0.02 0.01 1.   ]
▶ 0.2
```

왜 쓰나 정규화 결과처럼 소수점이 17자리까지 나오면 읽을 수가 없다. 보고서에 넣을 숫자는 보통 **소수 둘째 자리**까지가 관례다.

응용 `np.round(배열, 2)`와 `배열.round(2)`는 같은 일을 한다. 하지만 **파이썬 내장** `round()`는 배열에 못 쓴다 — 값 하나에만 쓸 수 있다. 강사도 이 구분을 짚었다. 통계 결과처럼 값 하나가 나오는 경우엔 `round(s.mean(), 2)`, 배열이면 `np.round(arr, 2)`다.

```
arr = np.array([1.2345, 2.6789])

print(np.round(arr, 2))      # 배열 → np.round
print(arr.round(2))         # 배열 → .round() 메서드
print(round(arr.mean(), 2))  # 값 하나 → 내장 round

# round(arr, 2) 는 TypeError
# type numpy.ndarray has no __round__ method
> [1.23 2.68]
> [1.23 2.68]
> 1.96
```

자주 하는 실수

`round(배열, 2)`처럼 내장 함수를 배열에 쓰면 `TypeError: numpy.ndarray has no __round__ method`가 난다. **배열이면** `np.round`, 값 하나면 `round`.

설비 적용

정규화에는 함정이 하나 있다. **최댓값이 이상치면 정규화 전체가 왜곡된다**. 위 예에서 2861 하나 때문에 나머지 다섯 값이 전부 0~0.1 구간에 눌러 버렸다. 실무에서는 이상치를 먼저 걸러낸 뒤 정규화하거나, 최댓값 대신 **99 퍼센타일**을 기준으로 삼는다. 오늘 배울 필터링이 그 전 단계다.

STEP 3

비교 연산과 불리언 배열

조건을 배열로 바꾸기

필터링의 첫 단계는 **값을 골라내는 것이 아니라 조건에 맞는 위치를 표시하는 것**이다. 이 중간 단계를 건너뛰고 이해하려 하면 `v[v > 85]`가 왜 그렇게 생겼는지 영영 안 잡힌다. 순서대로 가자.

3-1. 비교 연산 — 결과가 배열로 나온다

연산자 `v > 85` (비교 연산) 조건을 걸면 참거짓 배열이 만들어짐

정의 배열에 비교 연산자를 쓰면 **원래 배열과 같은 크기의 참거짓(bool) 배열**이 나온다. 조건에 맞는 위치만 True.

형태 배열 > 값 · 배열 < 값 · 배열 == 값 · 배열 != 값

```
import numpy as np

v = np.array([70, 95, 71, 88, 73])

print(v > 85)
print(v < 75)
print(v == 71)
print(v.dtype, (v > 85).dtype)

▶ [False True False True False]
▶ [ True False  True False  True]
▶ [False False  True False False]
▶ int64 bool
```

왜 쓰나 백만 개 값도 한 줄로 비교된다. **비교 연산도 벡터 연산**이라서 반복문이 필요 없다. 거대한 센서 데이터에서 조건에 맞는 위치를 순식간에 찾아낸다.

응용 결과의 dtype이 bool로 바뀌는 것에 주목하자. **원래 값이 사라지고 위치 정보만 남은 상태**다. 이 불리언 배열 자체가 다음 단계의 재료가 된다. 2차원 배열에도 똑같이 적용된다.


```
data = np.array([
    [4603, 2.0],
    [ 58, 5.0],
    [4967, 21.0]
])

print(data > 1000)
print()
print("회전수 열만:", data[:, 0] > 1000)
```

```
▶ [[ True False]
   [False False]
   [ True  True]]
▶ 회전수 열만: [ True False  True]
```

주의

같다 비교는 등호 두 개(==) 다. 등호 하나(=)는 대입이라 SyntaxError가 난다. 실수 비교에는 ==를 쓰지 않는 것이 좋다 — 부동소수점 오차 때문에 `0.1 + 0.2 == 0.3`이 False다.

3-2. 불리언 배열이란 — 골라낸 게 아니라 표시한 것

이 부분이 오늘의 핵심 개념이다. `v > 85`의 결과는 값을 골라낸 것이 아니다. 원래 자리에 참거짓만 그대로 표시된 상태다. 크기도 원래 배열과 똑같다.

```
두 단계 — ① 표시하고 ② 골라낸다
v = np.array([70, 95, 71, 88, 73])
mask = v > 85

print("원본 :", v)
print("표시 :", mask)
print("크기 :", v.size, mask.size)

# 아직 값을 고르지 않았다 — 다음 단계에서 고른다
print("골라냄:", v[mask])
```

```
▶ 원본 : [70 95 71 88 73]
▶ 표시 : [False  True False  True False]
▶ 크기 : 5 5
▶ 골라냄: [95 88]
```

① 비교 연산
`v > 85`

② 불리언 배열
[F T F T F]

③ 값 골라내기
`v[mask]`

비유

불리언 배열은 **형광펜으로 밑줄만 그어 둔 상태**다. 아직 아무것도 오려 내지 않았다. 종이는 그대로 있고 표시만 되어 있다. 다음 단계에서 밑줄 친 부분만 오려 내는 것이 Boolean indexing이다.

3-3. 불리언 배열의 합과 평균 — 개수와 비율 (실습 5)

여기서 재미있는 일이 벌어진다. 불리언 배열에 `.sum()`을 하면 **True의 개수**가, `.mean()`을 하면 **True의 비율**이 나온다. 파이썬이 True를 1, False를 0으로 취급하기 때문이다.

메서드 `.sum()` · `.mean()` (불리언 배열에) 조건을 만족하는 개수와 비율

정의 True는 1, False는 0으로 계산된다. 합계는 개수, 평균은 전체 대비 비율이 된다.

```
torque = np.array([42.8, 46.3, 49.4, 4.6, 41.9, 65.7, 40.2, 60.7])

high = torque > 50
print(high)

print("개수:", high.sum())
print("비율:", round(high.mean(), 2))
```

▶ [False False False False False True False True]
▶ 개수: 2
▶ 비율: 0.25

왜 쓰나 "전체의 몇 %가 기준을 넘었나"가 설비 분석의 표준 보고 형식이다. "경험상 위험하다"를 "데이터상 25%가 기준 초과"로 바꾸는 것 — 진짜 분석의 시작이다.

응용 강사가 든 비유가 좋았다 — "이 방에서 안경 쓴 사람 몇 명이지?" 먼저 세고, 그다음 "전체 중 몇 %인지"를 낸다. 코드로는 `.sum()`과 `.mean()` 두 줄이다. 실무 리포트 한 줄로 묶으면 아래처럼 된다.

```
torque = np.array([42.8, 46.3, 49.4, 4.6, 41.9, 65.7, 40.2, 60.7])
high = torque > 50
```

```
print(f"기준(50) 초과: {high.sum()}건 / 전체 {torque.size}건 "
      f"({high.mean() * 100:.1f}%)")
```

값까지 함께 보고 싶다면

```
print("초과 값:", torque[high])
```

▶ 기준(50) 초과: 2건 / 전체 8건 (25.0%)

▶ 초과 값: [65.7 60.7]

개념

.mean() 이 비율이 되는 이유는 (True 개수) ÷ (전체 개수) 이기 때문이다. 평균의 정의가 그대로 비율 계산이 된 것뿐이다.

주의

비율은 소수점이 길게 나올 수 있다. 0.25처럼 딱 떨어지면 다행이지만 0.3333333333333333이 나오면 읽기 어렵다. round(비율, 2)로 두 자리까지 끊어 주자. 강사도 "소수점 두 자리가 거의 국룰"이라고 했다.

3-4. 비교 연산자 전체와 실수 비교 주의

오늘 쓰는 비교 연산자는 파이썬에서 배운 것과 **완전히 같다**. 달라지는 건 결과가 값 하나가 아니라 **배열**이라는 점뿐이다.

연산자	뜻	예	결과
>	초과	v > 85	85보다 큰 자리 True
>=	이상	v >= 85	85 포함
<	미만	v < 75	75보다 작은 자리 True
<=	이하	v <= 75	75 포함
==	같다 (등호 두 개)	v == 71	정확히 71인 자리 True
!=	다르다	v != 71	71이 아닌 자리 True

자주 하는 실수

실수(float)에 ==를 쓰면 안 된다. 부동소수점 오차 때문에 0.1 + 0.2 == 0.3이 False다. 실수 비교에는 np.isclose()를 쓰거나 범위로 비교한다. 센서값은 거의 다 실수라서 실무에서 반드시 만나는 함정이다.

실수는 == 로 비교하지 않는다

```
import numpy as np
```

```
a = np.array([0.1 + 0.2, 0.3])  
print(a)
```

```
print(a[0] == a[1])          # 눈으로는 같아 보이는데
```

```
print(a[0] - a[1])          # 아주 미세한 차이가 있다
```

```
# 실수 비교는 isclose 로
```

```
print(np.isclose(a[0], a[1]))
```

```
# 또는 범위로
```

```
print(abs(a[0] - a[1]) < 1e-9)
```

```
▶ [0.3 0.3]
```

```
▶ False
```

```
▶ 5.551115123125783e-17
```

```
▶ True
```

```
▶ True
```

3-5. 부정(~)과 요약(any · all)

불리언 배열을 다루는 도구가 두 개 더 있다. 하나는 **뒤집기**, 다른 하나는 **한 줄로 요약하기**다. 다중 조건 오류 메시지가 친절하게 알려 주는 `a.any()` · `a.all()`이 바로 이것이다.

연산자 ~ (부정) 불리언 배열을 뒤집기

정의 True는 False로, False는 True로 바꾼다. 파이썬의 `not`에 해당하지만 **배열에는 ~**를 쓴다.

```
import numpy as np

v = np.array([70, 95, 71, 88, 73])
mask = v > 85

print(mask)
print(~mask)

print("초과 :", v[mask])
print("이하 :", v[~mask])
```

```
▶ [False True False True False]
▶ [ True False True False True]
▶ 초과 : [95 88]
▶ 이하 : [70 71 73]
```

왜 쓰나 조건을 두 번 쓰지 않고 **"나머지 전부"**를 뽑을 때 편하다. 조건이 복잡할수록 이득이 크다.

응용 조건이 길어지면 ~가 훨씬 읽기 좋다. `(rpm > 2000) | (torque < 10)`의 반대를 쓰려면 부등호를 전부 뒤집고 &로 바꿔야 하는데, `~(...)` 한 글자면 끝난다.

```
rpm = np.array([1551, 1408, 2861, 1410])
torque = np.array([42.8, 46.3, 4.6, 65.7])

danger = (rpm > 2000) | (torque < 10)

print("위험 시점:", rpm[danger])
print("정상 시점:", rpm[~danger])
```

```
▶ 위험 시점: [2861]
▶ 정상 시점: [1551 1408 1410]
```

주의

`not mask`처럼 쓰면 안 된다. `and`·`or`와 같은 이유로 `ValueError`가 난다. **배열에는 ~**.

메서드 `.any()` · `.all()` 불리언 배열을 참·거짓 하나로 요약

정의 `any()`는 하나라도 True면 True, `all()`은 전부 True여야 True.

```
rpm = np.array([4603, 58, 4967])

print("하나라도 이상?:", np.any(rpm < 1000))
print("전부 정상?    :", np.all(rpm > 1000))

# 메서드 형태로도 가능
print((rpm < 1000).any(), (rpm > 1000).all())
```

```
▶ 하나라도 이상?: True
▶ 전부 정상?    : False
▶ True False
```

왜 쓰나 "이 배치에 불량이 하나라도 있었나", "모든 시점이 정상 범위였나" 같은 판정 한 줄로 쓴다. 결과가 값 하나라서 if 문에 바로 넣을 수 있다.

응용 다중 조건 오류 메시지 Use `a.any()` or `a.all()`이 가리키는 것이 이 둘이다. 배열을 if 조건에 넣으면 파이썬이 "값이 여러 개인데 뭘 기준으로 판단하냐"고 되묻는 것인데, `any()`나 `all()`로 값 하나로 줄여 주면 해결된다.

```
rpm = np.array([4603, 58, 4967])

# x if rpm < 1000: → ValueError

# ✓ 값 하나로 줄인다
if (rpm < 1000).any():
    print("이상 시점 발견 — 점검 필요")
else:
    print("전 시점 정상")
```

```
▶ 이상 시점 발견 — 점검 필요
```

개념

`.sum()`이 "몇 개인가"를 묻는다면 `.any()`는 "있기는 한가", `.all()`은 "전부인가"를 묻는다. 세 메서드 모두 불리언 배열을 받아 값 하나로 줄여 준다.

STEP 4

값 골라내기

Boolean indexing과 np.where — 골라낼 것인가, 표시할 것인가

4-1. Boolean indexing — 조건에 맞는 값만 남기기

문법 `v[조건]` (Boolean indexing) 불리언 배열로 값만 골라내기

정의 대괄호 안에 조건을 직접 넣으면 **True** 자리의 값만 남고 나머지는 버려진다. 결과 크기가 줄어든다.

형태 배열[조건식]

```
import numpy as np

v = np.array([70, 95, 71, 88, 73])

print(v[v > 85])
print(v[v < 75])

# 두 단계로 나눠 쓴 것과 같다
mask = v > 85
print(v[mask])
```

- ▶ [95 88]
- ▶ [70 71 73]
- ▶ [95 88]

왜 쓰나 위험 수준을 넘은 센서값만 골라낼 때 쓰는 가장 기본 방법이다. 이상 징후 탐색의 첫 도구다.

응용 원리는 두 단계다 — ① **조건이 참거짓 배열을 만들고** ② **그 배열로 값을 골라낸다**. `v[v > 85]`는 이 두 단계를 한 줄로 압축한 표기일 뿐이다. 2차원에서는 조건에 맞는 값들이 1차원으로 퍼져서 나온다는 점을 주의하자.

```
data = np.array([
    [4603, 2.0],
    [ 58, 5.0],
    [4967, 21.0]
])

# 2차원에 조건을 걸면 결과는 1차원
print(data[data > 100])

# 보통은 열을 먼저 뽑고 조건을 건다
rpm = data[:, 0]
print("이상 회전수:", rpm[rpm < 1000])
```

▶ [4603. 4967.]

▶ 이상 회전수: [58.]

자주 하는 실수

`v[True]`처럼 조건 대신 `True`를 직접 넣으면 **값을 고르는 게 아니라 차원이 하나 늘어난다**. 강사 코드에도 이 예가 있다 — `v[True]`의 결과는 `[[70 95 71 88 73]]`으로 shape이 (1, 5)가 된다. 조건식을 넣어야 필터링이다.

4-2. np.where — 값을 바꾸되 크기는 유지

함수 `np.where()` 조건에 따라 값을 둘 중 하나로 바꾸기

정의 조건이 참인 자리는 두 번째 인자로, 거짓인 자리는 세 번째 인자로 바꾼다. **결과 크기는 원본 그대로**.

형태 `np.where(조건, 참일_때_값, 거짓일_때_값)`

```
v = np.array([70, 95, 71, 88, 73])

# 85 초과면 1(위험), 아니면 0(정상)
print(np.where(v > 85, 1, 0))

# 숫자 대신 글자도 가능
print(np.where(v > 85, "위험", "정상"))
```

▶ [0 1 0 1 0]

▶ ['정상' '위험' '정상' '위험' '정상']

왜 쓰나 모든 값을 유지하면서 위험·정상 꼬리표를 붙일 때 쓴다. 시점 순서를 그대로 두고 어느 시점이 위험했는지 표시해야 하는 경우가 실무에 많다.

응용 꼬리표가 숫자면 곧바로 통계로 이어진다. `np.where(...)`의 결과에 `.sum()`을 하면 위험 시점 개수가, `.mean()`을 하면 위험 비율이 나온다. **불리언 배열의 sum/mean과 같은 원리다.** 조건 자리에 배열 계산을 직접 넣을 수도 있다.

```
rpm = np.array([4603, 58, 4967, 1216, 4603])

flag = np.where(rpm < 1000, 1, 0)
print("위험 표시:", flag)
print("위험 건수:", flag.sum())
print("위험 비율:", round(flag.mean(), 2))

# 값을 보정하는 용도로도 쓴다 (하한 클리핑)
print("하한 보정:", np.where(rpm < 1000, 1000, rpm))
```

▶ 위험 표시: [0 1 0 0 0]
 ▶ 위험 건수: 1
 ▶ 위험 비율: 0.2
 ▶ 하한 보정: [4603 1000 4967 1216 4603]

개념

세 번째 인자에 원본 배열을 그대로 넣으면 **조건에 맞는 값만 바꾸고 나머지는 그대로 두는** 형태가 된다. 결측·이상값 보정에 자주 쓰는 패턴이다.

4-3. 둘을 가르는 기준 — 결과 크기

교안 p32가 이 둘을 나란히 놓고 비교한다. 선택 기준은 하나 — **결과 크기가 작아져도 되는가, 그대로 유지해야 하는가.**

구분		
하는 일	조건 맞는 값만 남김	모든 값 유지하고 바꿈
결과 크기	줄어듦 (5개 → 2개)	그대로 (5개 → 5개)
결과 예	[95 88]	[0 1 0 1 0]
언제 쓰나	위험값만 보고 싶을 때	전체에 꼬리표를 달고 싶을 때
이어지는 작업	골라낸 값들의 평균·개수	시점별 위험 표시, 시각화
시점 정보	사라짐 — 몇 번째였는지 모름	유지됨 — 위치가 그대로

같은 조건, 다른 결과

```
v = np.array([70, 95, 71, 88, 73])

picked = v[v > 85]          # 골라내기
flagged = np.where(v > 85, 1, 0) # 표시하기

print("골라냄:", picked, " · 크기", picked.size)
print("표시함:", flagged, " · 크기", flagged.size)

# 위치까지 알고 싶다면 np.where 를 인자 하나로
print("위험 위치:", np.where(v > 85)[0])
```

```
▶ 골라냄: [95 88] · 크기 2
▶ 표시함: [0 1 0 1 0] · 크기 5
▶ 위험 위치: [1 3]
```

설비 적용

설비 리포트에서는 둘 다 쓴다. "이상 시점이 몇 번째였는가"는 `np.where(조건)[0]`으로 위치를 뽑고, "이상 시점의 평균 회전수는 얼마였는가"는 `rpm[rpm < 1000].mean()`으로 값을 골라 낸다. 실습 9의 종합 분석이 정확히 이 두 가지를 묻는다.

4-4. np.where 중첩 — 등급을 세 단계로 나누기

`np.where`는 참·거짓 두 갈래만 처리한다. 그런데 실무에서는 **정상 / 주의 / 위험** 세 단계 이상이 필요한 경우가 많다. 이때는 거짓 자리에 다시 `np.where`를 넣는다.

중첩 np.where — 3단계 등급

```
import numpy as np

rpm = np.array([4603, 58, 4967, 1216, 2500, 4987])

# 1000 미만이면 "위험", 2000 미만이면 "주의", 나머지는 "정상"
grade = np.where(rpm < 1000, "위험",
                 np.where(rpm < 2000, "주의", "정상"))

print(rpm)
print(grade)

# 등급별 개수 세기
for g in ["위험", "주의", "정상"]:
    print(f"{g}: {(grade == g).sum()}건")
```

```
▶ [4603  58 4967 1216 2500 4987]
▶ ['정상' '위험' '정상' '주의' '정상' '정상']
▶ 위험: 1건
▶ 주의: 1건
▶ 정상: 4건
```

주의

중첩이 세 겹을 넘어가면 읽기가 급격히 나빠진다. 그때는 `np.select([조건들], [값들], default=...)`를 쓰는 편이 낫다. 오늘 수업 범위는 아니지만, 등급이 네 개 이상이면 그쪽을 찾아보자.

연결

마지막 줄의 `(grade == g).sum()`을 보자. **비교 연산으로 불리언 배열을 만들고 .sum()으로 개수를 센다** — **STEP 3에서 배운 그대로다. 오늘 배우는 조각들이 이렇게 계속 조합된다. 낱개로 외우지 말고 조합해서 쓰는 연습을 해야 하는 이유다.**

STEP 5

다중 조건

괄호와 & | — 초보자가 가장 많이 틀리는 곳

조건이 하나면 쉽다. 문제는 **두 개 이상을 묶을 때**다. 교안도 p34에서 "초보자가 가장 자주 틀리는 부분 — 괄호 누락"이라고 못 박는다. 규칙은 딱 두 가지뿐이다.

5-1. 두 가지 규칙 — 괄호로 묶고, & 또는 |로 연결

연산자 (조건1) & (조건2) 두 조건을 모두 만족 (그리고)

정의 각 조건을 반드시 괄호로 묶고 &로 연결한다. 둘 다 참인 위치만 True.

형태 배열[(조건1) & (조건2)] · 배열[(조건1) | (조건2)]

```
import numpy as np

v = np.array([70, 95, 71, 88, 73])

# 70 초과 그리고 90 미만
print(v[(v > 70) & (v < 90)])

# 70 이하 또는 90 이상
print(v[(v <= 70) | (v >= 90)])

▶ [71 88 73]
▶ [70 95]
```

왜 쓰나 실무 조건은 거의 항상 복합이다. "정상 범위"는 &(그리고), "여러 위험 신호 중 하나"는 |(또는) 를 쓴다.

응용 두 단계로 나눠 써도 결과는 같다. 오히려 처음엔 나눠 쓰는 편이 이해가 빠르다. 강사 코드도 두 방식을 나란히 보여 준다. 익숙해지면 한 줄로 합치면 된다.

```
v = np.array([70, 95, 71, 88, 73])
```

```
# 방법 1 - 두 단계로
step1 = v[v > 70]
step2 = step1[step1 < 90]
print(step2)
```

```
# 방법 2 - 한 줄로 (결과 동일)
print(v[(v > 70) & (v < 90)])
```

```
▶ [71 88 73]
▶ [71 88 73]
```

자주 하는 실수

괄호를 빠뜨리면 오류다. `v[v > 70 & v < 90]`은 `&`가 비교(`>`, `<`)보다 먼저 계산되려 해서 순서가 꼬인다.
`ValueError: The truth value of an array with more than one element is ambiguous.`

5-2. and · or 를 쓰면 왜 안 되는가

파이썬에서 배운 `and`, `or`를 그대로 쓰고 싶어지는데, 배열에는 안 된다. `and/or`는 값 하나가 참인지 거짓인지를 묻는 연산자라서, 값이 여러 개인 배열을 받으면 무엇을 답해야 할지 모른다.

세 가지 시도, 하나만 정답

```
v = np.array([70, 95, 71, 88, 73])
```

```
# x and 사용
# print(v[(v > 70) and (v < 90)])
# ValueError: The truth value of an array with more than one
# element is ambiguous. Use a.any() or a.all()
```

```
# x 괄호 누락
# print(v[v > 70 & v < 90])
# ValueError: 같은 오류
```

```
# ✓ 괄호 + &
print(v[(v > 70) & (v < 90)])
```

```
▶ [71 88 73]
```

상황	파이썬 기본	넘파이 배열	이유
그리고	<code>and</code>	<code>&</code> (각 조건 괄호)	배열은 원소별로 판단해야 함

상황	파이썬 기본	넘파이 배열	이유
또는	or	(각 조건 괄호)	같음
부정	not	~ (조건 괄호)	같음

개념

and는 "이 하나가 참인가" 를 묻고, &는 "각 자리마다 둘 다 참인가" 를 묻는다. 배열은 자리가 여러 개니까 &를 써야 한다. 오류 메시지의 "ambiguous(모호하다)"가 정확히 이 뜻이다 — 값이 다섯 개인데 그중 무엇을 기준으로 참거짓을 답하라는 건지 모르겠다는 것이다.

5-3. 실습 4 — 회전수 과다 또는 토크 과소

교안 실습 4는 |(또는) 을 쓰는 문제다. "회전수가 너무 높거나, 토크가 너무 낮으면 위험"이라는 조건이다. 두 배열의 같은 인덱스가 같은 시점을 가리킨다는 점이 핵심이다.

실습 4 — 두 센서를 함께 보기

```
import numpy as np
```

```
# rpm[i] 와 torque[i] 는 같은 시점의 값
rpm = np.array([1551, 1408, 1498, 2861, 1425, 1410])
torque = np.array([42.8, 46.3, 49.4, 4.6, 41.9, 65.7])
```

```
# 단일 조건 — 회전수 2000 초과
print("회전수 초과:", rpm[rpm > 2000])
```

```
# 다중 조건 — 회전수 과다 또는 토크 과소
danger = (rpm > 2000) | (torque < 10)
print("위험 시점 :", danger)
print("위험 건수 :", danger.sum())
```

```
▶ 회전수 초과: [2861]
▶ 위험 시점 : [False False False True False False]
▶ 위험 건수 : 1
```

설비 적용

3번 시점만 True다. 회전수 2861(과다)이면서 동시에 토크 4.6(과소)인 시점이 **우연히 딱 겹쳤다**. 회전은 빠르는데 힘은 안 걸린다 — 공구가 부러졌거나 소재를 헛도는 전형적인 신호다. 강사가 강조했듯 **필터링은 기술이 아니라 판단**이다. &와 | 중 무엇을 쓸지 정하는 순간, 이미 "무엇을 이상으로 볼 것인가"를 정의한 것이다.

STEP 6

기초 통계 ① 중심과 범위

평균의 합정과 중앙값의 강점

6-1. 합계와 평균

메서드 `.sum()` · `.mean()` 전체를 더하고, 개수로 나눈 대표값

정의 `sum()`은 모든 값의 합, `mean()`은 그 합을 개수로 나눈 평균.

형태 `배열.sum()` · `배열.mean()`

```
import numpy as np

s = np.array([70, 72, 71, 95, 73])

print(s.sum())
print(s.mean())

# 함수 형태로도 가능
print(np.sum(s), np.mean(s))
```

▶ 381
▶ 76.2
▶ 381 76.2

왜 쓰나 가장 먼저 구해 보는 대표값이다. "이 설비의 평상시 상태는 대략 얼마인가"에 답한다.

응용 그런데 위 결과를 자세히 보자. 다섯 값 중 넷은 70~73인데 **평균이 76.2**다. 대부분보다 위에 있다. 95 하나가 평균을 끌어올린 것이다. 평균만 보고 "이 설비는 76도 근처에서 돈다"고 하면 틀린 그림이다.

```
s = np.array([70, 72, 71, 95, 73])

print("값들 :", s)
print("평균 :", s.mean())
print("95 제외 :", s[s < 90].mean())
print("차이 :", round(s.mean() - s[s < 90].mean(), 2))
```

▶ 값들 : [70 72 71 95 73]
▶ 평균 : 76.2
▶ 95 제외 : 71.5
▶ 차이 : 4.7

개념

.sum()과 .mean()은 **불리언 배열에도 쓸 수 있다**(STEP 3 참고). 조건 배열이면 개수와 비율이 되고, 숫자 배열이면 합계와 평균이 된다. 같은 메서드가 자료에 따라 의미가 달라지는 것이다.

6-2. 평균의 약점 — 이상치 하나에 휘둘린다

평균은 **모든 값을 더해서** 계산한다. 그래서 유난히 큰 값 하나가 합계에 그대로 반영되고, 평균 전체를 끌어당긴다. 보조 교재가 든 예가 극적이다.

사례	구성	평균	중앙값	어느 쪽이 현실적인가
CEO 연봉 착시	직원 4명 3천만 + CEO 10억	2억 2천6백만	3천만	중앙값
설비 온도	70·72·71·95·73	76.2	72.0	중앙값
안정 설비	70·71·70·72·71	70.8	71.0	둘 다 비슷

개념

강사가 실제 사례로 든 이야기가 있다. **국회의원 재산 평균**을 낼 때 한 명이 수백억이면 전체 평균이 크게 올라가서, 나머지 대부분이 그 아래인데도 "다들 부자"라는 잘못된 그림이 나온다. 그래서 실무에서는 그런 극단값을 **양해를 구하고 통계에서 제외**하는 처리를 한다. 이것이 데이터 **클렌징(cleansing)** 이다.

6-3. 중앙값 — 줄 세웠을 때 한가운데

함수 np.median() 값을 줄 세웠을 때 한가운데 오는 값

정의 정렬한 뒤 가운데 위치의 값. 값이 짝수 개면 가운데 두 값의 평균.

형태 np.median(배열)


```
s = np.array([70, 72, 71, 95, 73])

print(np.median(s))
print(s.mean())

# 극단값을 더 키워도 중앙값은 그대로
s2 = np.array([70, 72, 71, 950, 73])
print(np.median(s2), s2.mean())
```

```
▶ 72.0
▶ 76.2
▶ 72.0 247.2
```

왜 쓰나 이상치에 흔들리지 않는다. 맨 끝 값이 95든 950이든 한가운데 위치는 변하지 않는다. 줄의 중앙은 극단값과 무관하기 때문이다.

응용 실무 기법이 하나 있다 — **평균과 중앙값을 함께 구해 비교하는 것**. 두 값이 크게 다르면 "이 데이터에 이상치가 있다"는 신호다. 계산 두 줄로 하는 **데이터 건강 점검**이다.

```
def health_check(arr, name):
    m, med = arr.mean(), np.median(arr)
    gap = abs(m - med)
    verdict = "이상치 의심" if gap > med * 0.05 else "정상"
    print(f"{name}: 평균 {m:.1f} · 중앙값 {med:.1f} · 차이 {gap:.1f} → {verdict}")

health_check(np.array([70, 71, 70, 72, 71]), "안정 설비")
health_check(np.array([70, 72, 71, 95, 73]), "이상 설비")
```

```
▶ 안정 설비: 평균 70.8 · 중앙값 71.0 · 차이 0.2 → 정상
▶ 이상 설비: 평균 76.2 · 중앙값 72.0 · 차이 4.2 → 이상치 의심
```

자주 하는 실수

중앙값만 `np.median(배열)` 형태의 함수다. `s.median()`은 존재하지 않아 `AttributeError`가 난다. 오늘 배우는 통계 중 유일한 예외라 시험에 나오기 좋다.

6-4. 최대·최소·범위

메서드 `.max()` · `.min()` 가장 큰 값·작은 값, 그 차이로 폭 파악

정의 배열에서 가장 큰 값과 가장 작은 값. 둘의 차이가 데이터가 퍼진 폭(범위)이다.

형태 `배열.max()` · `배열.min()` · `범위 = max - min`

```
s = np.array([70, 72, 71, 95, 73])
```

```
print(s.max(), s.min())  
print(s.max() - s.min())
```

```
# 위치까지 알고 싶으면
```

```
print("최댓값 위치:", s.argmax())  
print("최솟값 위치:", s.argmin())
```

- ▶ 95 70
- ▶ 25
- ▶ 최댓값 위치: 3
- ▶ 최솟값 위치: 0

왜 쓰나 범위가 크면 값이 넓게 퍼졌다는 뜻, 작으면 안정적으로 모여 있다는 뜻이다. 정상 범위를 넘은 최댓값은 "위험했던 순간"의 신호다.

응용 `.argmax()` / `.argmin()`은 값이 아니라 위치(인덱스)를 돌려준다. 강사가 다루지는 않았지만 실무에서는 이쪽을 훨씬 많이 쓴다 — "언제 최고였나"를 알아야 로그를 되짚어 볼 수 있기 때문이다.

```
rpm = np.array([4603, 58, 4967, 1216, 4603, 4987])
```

```
worst = rpm.argmin()  
print(f"최저 회전수 {rpm[worst]} - {worst}번째 시점")  
print(f"범위: {rpm.min()} ~ {rpm.max()} (폭 {rpm.max() - rpm.min()})")
```

- ▶ 최저 회전수 58 - 1번째 시점
- ▶ 범위: 58 ~ 4987 (폭 4929)

주의

최대·최소는 단 하나의 극단값이라 전체 판단에는 부족하다. 시작점일 뿐이고, 끝은 평균·표준편차와의 결합이다. 다음 STEP에서 이어진다.

6-5. 중심과 범위 정리

통계	형태	측면	한 줄 해석
합계	<code>s.sum()</code>	—	전체를 더한 값
평균	<code>s.mean()</code>	중심	대표값 · 이상치에 휘둘림
중앙값	<code>np.median(s)</code>	중심	이상치에 안 흔들림 · 함수 형태

통계	형태	측면	한 줄 해석
최대·최소	<code>s.max()</code> · <code>s.min()</code>	범위	위험했던 순간
범위	<code>s.max()</code> - <code>s.min()</code>	범위	퍼진 폭 · 안정도 힌트

6-6. 실제 데이터로 읽어 보기 — 중심과 범위만으로

표준편차 없이 중심과 범위만으로도 이 데이터에 문제가 있다는 걸 알 수 있다.

중심과 범위만 구해 봐도

```
import numpy as np

rpm = np.loadtxt("data/10_mct_tool.csv", delimiter=",",
                 skiprows=1, usecols=4, encoding="utf-8")

print(f"개수 : {rpm.size}")
print(f"평균 : {rpm.mean():.1f}")
print(f"중앙값 : {np.median(rpm):.1f}")
print(f"최대·최소: {rpm.max():.0f} · {rpm.min():.0f}")
print(f"최저 시점: {rpm.argmin()}번째")
```

▶ 개수 : 40
▶ 평균 : 4212.6
▶ 중앙값 : 4603.0
▶ 최대·최소: 4987 · 58
▶ 최저 시점: 20번째

관찰	숫자	무엇을 뜻하나
평균 < 중앙값	4212.6 < 4603.0	낮은 쪽에 이상치가 있다 — 평균을 아래로 끌어내림
최솟값이 58	중앙값의 1.3%	사실상 정지 — 정상 변동으로 설명 불가

설비 적용

평균이 중앙값보다 낮다는 것이 첫 번째 단서다. STEP 6-3의 건강 점검 그대로다. 그리고 최솟값 58은 중앙값 4603의 1.3%밖에 안 된다. 여기까지가 **중심과 범위만으로 알 수 있는 것이고**, "그래서 이게 얼마나 비정상인가"에 답하려면 다음 STEP의 표준편차가 필요하다.

STEP 7

기초 통계 ② 흩어짐

정상의 정의를 세우는 두 숫자

평균과 중앙값은 **중심**을 알려 준다. 하지만 중심만으로는 부족하다. 평균이 똑같이 70도인 설비 둘이 있어도, 하나는 69~71에서 조용히 돌고 다른 하나는 55~85를 오간다면 완전히 다른 설비다. 그 차이를 재는 것이 **분산**과 **표준편차**다.

7-1. 분산 — 흩어진 정도를 하나의 숫자로

메서드 `.var()` 값들이 평균에서 흩어진 정도

정의 각 값과 평균의 차이를 제곱해서 모두 더한 뒤 개수로 나눈 값. 작으면 안정, 크면 불안정.

형태 배열 `.var()`

```
import numpy as np

stable = np.array([70, 71, 70, 72, 71])
unstable = np.array([60, 85, 65, 95, 70])

print("평균:", stable.mean(), unstable.mean())
print("분산:", round(stable.var(), 1), round(unstable.var(), 1))
```

▶ 평균: 70.8 75.0

▶ 분산: 0.6 170.0

왜 쓰나 정상 설비는 분산이 작다. 고장 징후가 생기면 값이 요동쳐 분산이 커진다. 분산이 갑자기 커지면 안정성 점검이 필요하다는 신호다.

응용 왜 제곱을 하는지 알아 두면 좋다. **차이를 그냥 더하면 +와 -가 상쇄돼 항상 0이 된다.** 제곱해서 음수를 없앤 뒤 더하는 것이다. 아래는 넘파이 없이 손으로 계산해 검산한 것이다.

```

a = np.array([60, 85, 65, 95, 70])
mu = a.mean()

print("평균 :", mu)
print("편차 :", a - mu)
print("편차 합 :", (a - mu).sum(), " ← 항상 0")
print("편차 제곱 :", (a - mu) ** 2)
print("분산(수동):", ((a - mu) ** 2).mean())
print("분산(넘파이):", a.var())

```

```

▶ 평균 : 75.0
▶ 편차 : [-15.  10. -10.  20.  -5.]
▶ 편차 합 : 0.0 ← 항상 0
▶ 편차 제곱 : [225. 100. 100. 400.  25.]
▶ 분산(수동): 170.0
▶ 분산(넘파이): 170.0

```

자주 하는 실수

분산의 단위가 어색하다. 온도의 분산은 "도의 제곱($^{\circ}\text{C}^2$)"이다. 제곱해서 구했기 때문이다. 그래서 실무에서는 다음 절의 표준편차를 훨씬 자주 쓴다.

7-2. 표준편차 — 원래 단위로 되돌리기

메서드 `.std()` 분산의 제곱근 — 원래 단위로 흩어진 정도

정의 분산에 제곱근을 씌워 원래 단위로 되돌린 값. "평균에서 대략 이만큼 흩어져 있다"로 바로 읽힌다.

형태 배열.std()

```

s = np.array([70, 72, 71, 95, 73])

print("분산 :", round(s.var(), 2))
print("표준편차:", round(s.std(), 2))

# 표준편차는 분산의 제곱근
print("검산 :", round(np.sqrt(s.var()), 2))

```

```

▶ 분산 : 89.36
▶ 표준편차: 9.45
▶ 검산 : 9.45

```

왜 쓰나 해석이 바로 된다. 분산 89.36은 "도의 제곱"이라 감이 안 오지만, 표준편차 9.45는 "값들이 평균에서 약 9.45도 흩어져 있다" 로 그대로 읽힌다. 그래서 실무에서 분산보다 훨씬 자주 쓴다.

응용 평균과 표준편차 두 숫자가 있으면 "정상의 정의" 를 세울 수 있다. 평균이 같아도 표준편차가 다르면 전혀 다른 설비다. 이 두 숫자의 시간적 변화를 추적하는 것이 예지보전의 기본 아이디어다.

```
A = np.array([70, 71, 70, 69, 70]) # 설비 A
B = np.array([55, 85, 60, 90, 60]) # 설비 B

for name, arr in [("설비 A", A), ("설비 B", B)]:
    m, sd = arr.mean(), arr.std()
    print(f"{name}: 평균 {m:.1f}도 · 표준편차 {sd:.1f}도 "
          f"→ 정상 범위 {m - sd:.1f} ~ {m + sd:.1f}")
```

- ▶ 설비 A: 평균 70.0도 · 표준편차 0.6도 → 정상 범위 69.4 ~ 70.6
- ▶ 설비 B: 평균 70.0도 · 표준편차 14.1도 → 정상 범위 55.9 ~ 84.1

개념

평균만 보면 두 설비가 똑같다. 표준편차를 함께 봐야 B의 불안정이 드러난다. 평균은 중심, 표준편차는 흩어짐 — 둘을 함께 봐야 데이터 모습이 머릿속에 그려진다.

7-3. 3-시그마 규칙 — 이상 탐지의 표준 기법

보조 교재 「쉬운 수학 가이드 PART 2」가 다루는 내용이다. 강사도 정규분포를 길게 설명하며 이 개념을 깔았다. **공장 설비의 정상 데이터는 대부분 종 모양의 정규분포를 따른다.** 그리고 통계적으로 **평균에서 표준편차의 3배(3σ) 이상 벗어나는 데이터는 0.3% 미만**이다. 그러니 그 구역에 들어오면 "이상 징후"로 판단한다.

개념 3-시그마 (3σ) 규칙 평균 $\pm 3 \times$ 표준편차 밖이면 이상

정의 정상 범위를 평균 - $3 \times$ 표준편차 ~ 평균 + $3 \times$ 표준편차로 정의하고, 그 밖의 값을 이상치로 본다.

형태 `anomalies = arr[(arr < mean - 3*std) | (arr > mean + 3*std)]`

```
import numpy as np

rpm = np.loadtxt("data/10_mct_tool.csv", delimiter=",",
                 skiprows=1, usecols=4, encoding="utf-8")

mu, sd = rpm.mean(), rpm.std()
print(f"평균 {mu:.1f} · 표준편차 {sd:.1f}")
print(f"정상 범위: {mu - 3*sd:.1f} ~ {mu + 3*sd:.1f}")

anomalies = rpm[(rpm < mu - 3*sd) | (rpm > mu + 3*sd)]
print("이상치:", anomalies)
```

- ▶ 평균 4212.6 · 표준편차 1144.9
- ▶ 정상 범위: 777.8 ~ 7647.4
- ▶ 이상치: [58.]

왜 쓰나 기준을 손으로 정하지 않고 데이터가 스스로 정하게 한다. "회전수 1000 미만이면 이상"처럼 사람이 정한 임계치는 설비가 바뀌면 다시 정해야 하지만, 3σ 는 어떤 설비에도 그대로 적용된다.

응용 실무에서는 2σ (약 95%)와 3σ (약 99.7%)를 상황에 따라 골라 쓴다. **2σ 는 민감하게, 3σ 는 확실한 것만 잡는다.** 오탐(false alarm)을 줄이려면 3σ , 놓치는 걸 줄이려면 2σ 다.

```
for k in (2, 3):
    lo, hi = mu - k*sd, mu + k*sd
    hit = rpm[(rpm < lo) | (rpm > hi)]
    print(f"{k}σ 범위 {lo:8.1f} ~ {hi:8.1f} → 이상 {hit.size}건 {hit}")
```

- ▶ 2σ 범위 1922.9 ~ 6502.3 → 이상 4건 [58. 1216. 1217. 1241.]
- ▶ 3σ 범위 777.8 ~ 7647.4 → 이상 1건 [58.]

설비 적용

2σ 로 보면 4건(58, 1216, 1217, 1241), 3σ 로 보면 1건(58)이다. **58은 누가 봐도 이상**이고, 1200대는 "정상 저속 구간일 수도 있다". 어느 쪽을 이상으로 볼지는 **현장 지식**이 정한다. 통계는 후보를 좁혀 줄 뿐이다.

주의

넘파이 `.var()`·`.std()`는 기본이 **모집단 기준**(전체를 다 가진 경우)이다. 표본에서 모집단을 추정하려면 `ddof=1`을 준다. 위 예에서 `s.var()`는 89.36이지만 `s.var(ddof=1)`은 111.7이다. 수업 범위는 아니지만, 통계 수업이나 다른 라이브러리(pandas)와 값이 안 맞을 때 이것이 원인인 경우가 많다 — **pandas의 `.std()`는 기본이 `ddof=1`이다.**

STEP 8

axis

통계를 어느 방향으로 낼 것인가

1차원 배열이면 `.mean()` 하나로 끝난다. 그런데 **2차원 표**에서는 문제가 생긴다. 회전수와 토크가 한 표에 들어 있는데 그냥 `.mean()`을 하면 **회전수와 토크를 섞어서** 평균을 내 버린다. 아무 의미가 없는 숫자다.

8-1. 문제 상황 — 섞어서 평균을 내면

axis 없이 평균을 내면

```
import numpy as np

mat = np.array([
    [70, 2.1], # 시점 0 - 온도 70, 진동 2.1
    [72, 2.3]  # 시점 1 - 온도 72, 진동 2.3
])

print(mat.mean()) # 온도와 진동을 섞은 평균 - 무의미
> 36.6
```

36.6이라는 숫자는 온도도 진동도 아니다. **네 값을 다 더해서 4로 나눈 것뿐이다.** 온도는 온도끼리, 진동은 진동끼리 따로 계산해야 한다. 그 방향을 정하는 것이 **axis**다.

키워드 `axis=0` · `axis=1` 통계를 낼 방향 지정

정의 `axis=0`은 **행을 따라 아래로**(열별 = 센서별), `axis=1`은 **열을 따라 옆으로**(행별 = 시점별) 계산한다. 생략하면 전체.

형태 `배열.mean(axis=0)` · `배열.mean(axis=1)` · `배열.mean()`

```
mat = np.array([
    [70, 2.1],
    [72, 2.3]
])

print(mat.mean()) # 전체 평균
print(mat.mean(axis=0)) # 열별 = 센서별
print(mat.mean(axis=1)) # 행별 = 시점별
> 36.6
> [71.  2.2]
> [36.05 37.15]
```


왜 쓰나 설비 분석에서는 `axis=0`(센서별)이 압도적으로 자주 쓰인다. 열 하나가 센서 하나이므로, 센서별 평균·표준편차가 곧 각 센서의 정상 기준이 된다.

응용 헛갈리면 결과 개수로 구분하면 확실하다. 2행 3열 배열에서 `axis=0`은 값 3개(열 수만큼), `axis=1`은 값 2개(행 수만큼)가 나온다. "사라지는 축의 번호를 쓴다"고 외워도 된다 — `axis=0`은 0번 축(행)이 뭉개져 사라진다.

```
b = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print("shape:", b.shape)          # (2, 3)

print("axis=0:", b.mean(axis=0), "→ 값 3개 (열 수)")
print("axis=1:", b.mean(axis=1), "→ 값 2개 (행 수)")
print("생략 :", b.mean(), "→ 값 1개")
```

```
▶ shape: (2, 3)
▶ axis=0: [2.5 3.5 4.5] → 값 3개 (열 수)
▶ axis=1: [2. 5.] → 값 2개 (행 수)
▶ 생략 : 3.5 → 값 1개
```

개념

`axis`는 `mean`뿐 아니라 `sum`·`max`·`min`·`std`·`var` 전부에 쓸 수 있다. 형태가 같으니 하나만 익히면 나머지는 그대로 적용된다.

8-2. 실습 6 — 센서별 기초 통계

교안 실습 6이 정확히 이 문제다. "표 모양 데이터에서 센서별(열별) 통계를 계산하라." 강사도 "이건 사실 `axis`에 대한 이야기"라고 짚었다.

실습 6 — axis=0 으로 센서별 통계

```
import numpy as np

# 행=설비, 열=(회전수, 토크)
data6 = np.array([
    [1551, 42.8],
    [1408, 46.3],
    [1498, 49.4],
    [2861, 4.6]
])

print("전체 평균(무의미):", data6.mean())
print("센서별 평균 :", np.round(data6.mean(axis=0), 2))
print("센서별 표준편차 :", np.round(data6.std(axis=0), 2))

rpm_mean, torque_mean = data6.mean(axis=0)
print(f"회전수 평균 {rpm_mean:.1f} · 토크 평균 {torque_mean:.1f}")
```

▶ 전체 평균(무의미): 932.6375
▶ 센서별 평균 : [1829.5 35.78]
▶ 센서별 표준편차 : [597.73 18.15]
▶ 회전수 평균 1829.5 · 토크 평균 35.8

지정	방향	결과 의미	2행 3열에서 결과 개수
axis=0	행을 따라 아래로 ↓	열별 = 센서별 통계 (최빈출)	3개 (열 수)
axis=1	열을 따라 옆으로 →	행별 = 시점별 통계	2개 (행 수)
생략	전부	전체 하나의 값	1개

설비 적용

회전수 평균 1829.5에 표준편차 597.73 — **표준편차가 평균의 3분의 1**이나 된다. 2861 하나 때문이다. 토크도 평균 35.78에 표준편차 18.15로 흔들림이 크다. 두 숫자를 같이 읽으면 **"이 설비 그룹은 편차가 커서 하나의 기준으로 묶기 어렵다"**는 결론이 나온다. 통계는 답이 아니라 다음 질문을 만들어 준다.

연결

강사가 덧붙인 말이 중요하다 — **"넘파이에서는 axis가 그렇다는 예시이고, 판다스로 넘어가면 덜 중요해질 수 있다."** 판다스는 열에 이름이 붙어서 `df["회전수"].mean()`처럼 이름으로 부르면 되기 때문이다. 그래도 axis 개념 자체는 판다스에서도 계속 쓰이므로, 지금 방향 감각을 잡아 두면 다음 주가 편하다.

STEP 9

파일에서 배열로

np.loadtxt와 종합 분석 — 다섯 단계의 완성

지금까지는 배열을 손으로 만들어 썼다. 실무는 다르다. **데이터는 파일로 온다**. 13일차에 배운 `open()` 과 `csv` 모듈로도 읽을 수 있지만, 숫자만 다룰 거라면 넘파이가 훨씬 짧은 길을 제공한다.

9-1. np.loadtxt — CSV를 곧바로 배열로

함수 `np.loadtxt()` 텍스트·CSV 파일을 배열로 읽기

정의 구분자로 나뉜 숫자 텍스트 파일을 넘파이 배열로 바로 읽어 들인다.

형태 `np.loadtxt(경로, delimiter=",", skiprows=1, usecols=열번호, encoding="utf-8")`

```
import numpy as np

# 회전수(4번 열)만 읽기
rpm = np.loadtxt("data/10_mct_tool.csv", delimiter=",",
                 skiprows=1, usecols=4, encoding="utf-8")

print(rpm.shape, rpm.dtype)
print(rpm[:6])

▶ (40,) float64
▶ [4603. 4603. 4967. 4603. 4603. 4603.]
```

왜 쓰나 13일차의 `open()` + `csv.reader()` + `float()` 변환 세 단계가 **한 줄로 압축**된다. 숫자만 있는 파일이면 이쪽이 훨씬 짧고 빠르다.

응용 인자 하나하나가 다 이유가 있다. 특히 `usecols`에 **튜플을 주면 여러 열**을 한 번에 읽어 2차원 배열이 된다. 실습 9가 이 형태다.

```
# 회전수(4)와 토크(5) 두 열을 함께
data = np.loadtxt("data/10_mct_tool.csv", delimiter=",",
                  skiprows=1, usecols=(4, 5), encoding="utf-8")

print(data.shape, data.dtype)
print(data[:3])

# 열 분리는 슬라이싱으로 (15일차 내용)
rpm, torque = data[:, 0], data[:, 1]
print("회전수 앞 3개:", rpm[:3])

▶ (40, 2) float64
▶ [[4603.    2.]
   [4603.    2.]
   [4967.   21.]]
▶ 회전수 앞 3개: [4603. 4603. 4967.]
```

자주 하는 실수

skiprows=1을 빼면 헤더(공구번호, 사이클타임, ...)를 숫자로 바꾸려다 `ValueError: could not convert string to float`가 난다. 첫 줄이 열 이름이면 **반드시 건너뛰다**.

9-2. loadtxt 인자 정리

인자	뜻	예	빼먹으면
delimiter	값을 나누는 구분자	","	공백 기준으로 읽어 오류
skiprows	맨 위 몇 줄을 건너뛰는지	1	헤더를 숫자로 바꾸려다 오류
usecols	읽을 열 번호 (0부터)	4 또는 (4, 5)	전체 열을 읽음
encoding	파일 인코딩	"utf-8"	한글 헤더에서 깨질 수 있음
unpack	전치(transpose) 여부	True	기본 False — 표 모양 그대로

주의

usecols 번호는 **0부터** 센다. 10_mct_tool.csv의 열은 공구번호(0) · 사이클타임(1) · 온도(2) · 전류(3) · 회전수(4) · 토크(5) 이므로 회전수가 4, 토크가 5다. 엑셀에서 보이는 "다섯 번째 열"이 코드로는 4번이라는 점을 매번 확인하자.

9-3. unpack=True — 표를 눕히기

unpack=True를 주면 결과가 **전치(transpose)** 된다. (40, 2)가 (2, 40)이 되는 것이다. 열을 변수로 바로 받고 싶을 때 편하지만, shape이 뒤집히므로 이후 코드가 달라진다.

unpack 유무에 따른 차이

```
# 기본 — 표 모양 그대로 (행=시점, 열=센서)
data = np.loadtxt("data/10_mct_tool.csv", delimiter="," ,
                  skiprows=1, usecols=(4, 5), encoding="utf-8")
print("기본      :", data.shape)
print("회전수 뽑기:", "data[:, 0]")

# unpack=True — 전치됨 (행=센서, 열=시점)
u = np.loadtxt("data/10_mct_tool.csv", delimiter="," ,
               skiprows=1, usecols=(4, 5), encoding="utf-8", unpack=True)
print("unpack      :", u.shape)
print("회전수 뽑기:", "u[0]")

# 가장 깔끔한 사용법 — 바로 두 변수로 받기
rpm, torque = np.loadtxt("data/10_mct_tool.csv", delimiter="," ,
                         skiprows=1, usecols=(4, 5), encoding="utf-8",
                         unpack=True)
print("회전수:", rpm[:3], " · 토크:", torque[:3])
```

```
▶ 기본      : (40, 2)
▶ 회전수 뽑기: data[:, 0]
▶ unpack      : (2, 40)
▶ 회전수 뽑기: u[0]
▶ 회전수: [4603. 4603. 4967.] · 토크: [2. 2. 21.]
```

자주 하는 실수

교안 실습 9는 "shape과 dtype으로 구조 확인" 을 요구하고 예상 결과가 (40, 2)다. unpack=True를 쓰면 (2, 40)이 나와서 예상과 달라진다. 틀린 코드는 아니지만 교안이 의도한 형태가 아니다. unpack=True를 쓸 거면 반드시 두 변수로 받아 rpm, torque = ... 형태로 써야 의미가 산다.

9-4. 실습 9 — 다섯 단계를 하나의 흐름으로

오늘의 마지막이자 넘파이 단원 전체의 종합이다. 불러오기 → 구조 확인 → 추출 → 필터링 → 통계 요약, 다섯 단계를 한 파일에 담는다.

실습 9 — 종합 분석

```
import numpy as np

# ① 불러오기
data = np.loadtxt("data/10_mct_tool.csv", delimiter=",",
                  skiprows=1, usecols=(4, 5), encoding="utf-8")

# ② 구조 확인
print("shape:", data.shape, " · dtype:", data.dtype)

# ③ 추출 — 회전수 열만
rpm = data[:, 0]

# ④ 필터링 — 1000 미만인 이상 시점
anomaly = rpm[rpm < 1000]

# ⑤ 통계 요약
print("이상 시점:", anomaly)
print("개수   :", anomaly.size)
print("평균   :", round(anomaly.mean(), 1))
print("전체 평균:", round(rpm.mean(), 1))
```

```
▶ shape: (40, 2) · dtype: float64
▶ 이상 시점: [58.]
▶ 개수   : 1
▶ 평균   : 58.0
▶ 전체 평균: 4212.6
```

① 불러오기
np.loadtxt



② 구조 확인
shape · dtype



③ 추출
data[:, 0]



④ 필터링
rpm[rpm<1000]



⑤ 통계
size · mean

설비 적용

결과를 읽어 보자. 전체 평균 회전수는 4212.6인데 **이상 시점의 회전수는 58**이다. 정상의 1.4%밖에 안 된다. 사실상 **공구가 멈춘 순간**이다. 40개 시점 중 딱 한 번 — 이런 순간을 놓치지 않고 코드로 잡아내는 것이 예지보전의 출발점이다. 그리고 이 흐름이 몸에 배면, 데이터가 CSV든 데이터베이스든 어떤 형태로 와도 똑같이 다룰 수 있다.

치트시트

오늘 나온 전부 — 한 장 요약

A-1. 벡터 연산

종류	표현	예	조건
요소별	<code>a + b</code> · <code>a * b</code>	<code>[1,2] + [10,20] → [11,22]</code>	두 배열 크기가 같아야
스칼라	<code>a * 2</code> · <code>a + 5</code>	<code>c * 1.8 + 32</code> 화씨 변환	크기 무관
브로드캐스팅	<code>table - base</code>	<code>(2,2) - (2,) → (2,2)</code>	작은 쪽이 1이거나 같아야
정규화	<code>(x-min)/(max-min)</code>	0~1 범위로 맞추기	최댓값이 이상치면 왜곡
반올림	<code>np.round(a, 2)</code>	<code>a.round(2)</code> 도 가능	값 하나는 내장 round

A-2. 조건 필터링 네 가지

방법	표현	하는 일	결과 크기
비교 연산	<code>v > 85</code>	참거짓 판단 → 불리언 배열	그대로
불리언 인덱싱	<code>v[v > 85]</code>	조건 맞는 값만 골라내기	줄어듦
<code>np.where</code>	<code>np.where(v>85, 1, 0)</code>	조건에 따라 값 바꾸기	그대로
다중 조건	<code>(a) & (b)</code> · <code>(a) (b)</code>	조건 묶기 — 괄호 필수	경우별
개수 · 비율	<code>mask.sum()</code> · <code>mask.mean()</code>	True 개수 / True 비율	값 하나
위치 찾기	<code>np.where(v>85)[0]</code>	조건 맞는 인덱스	줄어듦

A-3. 기초 통계

통계	형태	측면	실무 해석
합계	<code>s.sum()</code>	—	전체 누적량
평균	<code>s.mean()</code>	중심	평상시 상태 · 이상치에 휘둘림
중앙값	<code>np.median(s)</code> ← 함수!	중심	이상치에 안 흔들림

통계	형태	측면	실무 해석
최대 · 최소	<code>s.max()</code> · <code>s.min()</code>	범위	위험했던 순간
범위	<code>s.max()</code> - <code>s.min()</code>	범위	퍼진 폭
분산	<code>s.var()</code>	흩어짐	제공 단위 — 해석 어려움
표준편차	<code>s.std()</code>	흩어짐	원래 단위 — 실무 표준
위치	<code>s.argmax()</code> · <code>s.argmin()</code>	—	언제 최고·최저였나

A-4. axis와 파일 읽기

표현	뜻	결과	비고
<code>arr.mean()</code>	전체 평균	값 하나	2차원에선 대개 무의미
<code>arr.mean(axis=0)</code>	열별 = 센서별	열 수만큼	실무 최빈출
<code>arr.mean(axis=1)</code>	행별 = 시점별	행 수만큼	시점 통합 지표
<code>np.loadtxt(경로, ...)</code>	CSV를 배열로	1차원 또는 2차원	<code>delimiter</code> · <code>skiprows</code> · <code>usecols</code>
<code>unpack=True</code>	전치해서 읽기	<code>(40, 2) → (2, 40)</code>	두 변수로 받을 때만

A-5. 3-시그마 이상 탐지 템플릿

그대로 가져다 쓸 수 있는 형태

```
import numpy as np

def find_anomalies(arr, k=3):
    """평균 ± k×표준편차 밖의 값을 이상치로 돌려준다."""
    mu, sd = arr.mean(), arr.std()
    lo, hi = mu - k * sd, mu + k * sd
    mask = (arr < lo) | (arr > hi)
    return arr[mask], np.where(mask)[0], (lo, hi)

rpm = np.loadtxt("data/10_mct_tool.csv", delimiter=",",
                 skiprows=1, usecols=4, encoding="utf-8")

values, positions, (lo, hi) = find_anomalies(rpm, k=3)
print(f"정상 범위: {lo:.1f} ~ {hi:.1f}")
print(f"이상치 {values.size}건: {values} (위치 {positions})")
```

▶ 정상 범위: 777.8 ~ 7647.4

▶ 이상치 1건: [58.] (위치 [20])

A-6. 자주 쓰는 조합 패턴 다섯 가지

오늘 배운 조각들을 실무에서 어떻게 붙여 쓰는지 모아 두었다. 그대로 복사해 쓰면 된다.

복붙해서 쓰는 다섯 패턴

```
import numpy as np

arr = np.array([4603, 58, 4967, 1216, 2500, 4987])
THRESHOLD = 2000

# ① 조건 만족 건수와 비율
mask = arr < THRESHOLD
print(f"① {mask.sum()}건 / {arr.size}건 ({mask.mean()*100:.1f}%)")

# ② 조건 만족 값들의 통계
sel = arr[mask]
print(f"② 평균 {sel.mean():.1f} · 최소 {sel.min():.0f}")

# ③ 조건 만족 위치(시점)
print(f"③ 위치 {np.where(mask)[0]}")
```

```
# ④ 위험·정상 꼬리표
print(f"④ {np.where(mask, 1, 0)}")

# ⑤ 정상 범위 벗어난 값 (3시그마)
mu, sd = arr.mean(), arr.std()
print(f"⑤ {arr[(arr < mu-3*sd) | (arr > mu+3*sd)]}")
```

- ▶ ① 2건 / 6건 (33.3%)
- ▶ ② 평균 637.0 · 최소 58
- ▶ ③ 위치 [1 3]
- ▶ ④ [0 1 0 1 0 0]
- ▶ ⑤ []

주의

⑤가 빈 배열 []인 것에 주목하자. 값이 6개뿐이라 표준편차가 크게 잡혀서 **3σ 범위 안에 전부 들어와 버렸다**. 3-시그마는 데이터가 충분히 많을 때 의미가 있다. 값이 적으면 규칙 기준(`arr < 2000`)을 쓰는 편이 낫다. **기법에는 각각 쓸 수 있는 상황이 있다.**

A-7. 한 줄로 답하기 — 상황별 코드

하고 싶은 것	코드 한 줄
0~1 범위로 정규화	<code>(a - a.min()) / (a.max() - a.min())</code>
기준 초과 값만 보기	<code>a[a > 기준]</code>
기준 초과 건수	<code>(a > 기준).sum()</code>
기준 초과 비율	<code>(a > 기준).mean()</code>
기준 초과 위치(시점)	<code>np.where(a > 기준)[0]</code>
위험 1 · 정상 0 꼬리표	<code>np.where(a > 기준, 1, 0)</code>
두 조건을 모두 만족	<code>a[(조건1) & (조건2)]</code>
조건의 반대	<code>a[~(조건)]</code>
하나라도 이상인가	<code>(a < 기준).any()</code>
이상치에 안 흔들리는 중심	<code>np.median(a)</code>
센서별(열별) 평균	<code>data.mean(axis=0)</code>
정상 범위 (3시그마)	<code>a.mean() - 3*a.std() ~ a.mean() + 3*a.std()</code>
CSV 한 열만 읽기	<code>np.loadtxt(경로, delimiter=",", skiprows=1, usecols=4)</code>

오류·함정 사전

✗ 이렇게 하면 → 무슨 일이 → ✓ 바로잡기

B-1. 오늘 만나기 쉬운 오류 10가지

✗ 이렇게 하면	무슨 일이 일어나	✓ 바로잡기
<code>v[v > 70 & v < 90]</code>	<code>ValueError: truth value ... ambiguous</code>	<code>v[(v > 70) & (v < 90)]</code> 괄호
<code>v[(a) and (b)]</code>	<code>ValueError: 같은 오류</code>	<code>& · · ~</code> 를 쓴다
<code>s.median()</code>	<code>AttributeError: no attribute 'median'</code>	<code>np.median(s)</code> 함수 형태
<code>round(배열, 2)</code>	<code>TypeError: no __round__ method</code>	<code>np.round(배열, 2)</code>
<code>np.array([1,2,3]) + np.array([1,2,3,4,5])</code>	<code>ValueError: could not be broadcast</code>	두 배열 크기를 맞춘다
<code>v[True]</code>	오류 없이 차원이 늘어남 → <code>shape (1,5)</code>	조건식을 넣는다 — <code>v[v > 85]</code>
<code>np.loadtxt(경로) (skiprows 없이)</code>	<code>could not convert string to float</code>	<code>skiprows=1</code> 헤더 건너뛰기
<code>usecols=5</code> 로 회전수 읽기	5번은 토크 — 엉뚱한 열	<code>usecols=4</code> 0부터 센다
<code>data.mean()</code> (2차원에)	센서를 섞은 무의미한 값	<code>data.mean(axis=0)</code> 센서별
<code>unpack=True</code> 쓰고 <code>data[:, 0]</code>	<code>shape</code> 이 (2,40)이라 엉뚱한 값	<code>rpm, torque = np.loadtxt(...)</code>

B-2. 제출 코드 교정 — 실제로 실행해 확인한 것

Practice/10_02_example.py와 Python/23_Statistics.py를 실제로 돌려 보고 정리했다. 오류가 나는 코드는 없었다. 다만 교안 요구와 어긋나거나 넘파이답지 않은 부분이 몇 군데 있다.

① 실습 2 — 요구한 것과 다른 것을 했다

교안 실습 2는 "이차원 배열을 준비하고 행 인덱싱·열 인덱싱으로 추출"을 요구한다. 제출 코드는 CSV를 `loadtxt`로 읽어 출력만 하고 행·열 추출을 하지 않았다. 파일 읽기는 실습 7의 내용이다.

Practice/10_02_example.py 실습 2

```
# x 제출 코드 — 읽어서 출력만
df = np.loadtxt(f, encoding="utf-8", delimiter=",", skiprows=1, usecols=(4, 5))
print(f"df:\n{df}")
```

✓ 요구사항 — 행·열 추출까지

```
data = np.loadtxt("Data/10_mct_tool.csv", delimiter=",",
                  skiprows=1, usecols=(4, 5), encoding="utf-8")
```

```
print("2행(설비 3호) :", data[2])          # 행 추출
```

```
print("회전수 열 :", data[:, 0][:5])      # 열 추출
```

```
print("토크 열 :", data[:, 1][:5])
```

▶ 2행(설비 3호) : [4967. 21.]

▶ 회전수 열 : [4603. 4603. 4967. 4603. 4603.]

▶ 토크 열 : [2. 2. 21. 2. 3.]

② 실습 3 · 5 · 6 미제출

실습	교안 요구	왜 중요한가
실습 3	정규화 — 회전수를 0~1로	브로드캐스팅의 대표 실전 · 머신러닝 전처리 필수
실습 5	조건별 개수와 비율	불리언 배열의 <code>.sum()</code> · <code>.mean()</code> — 오늘의 하이라이트
실습 6	axis로 센서별 통계	2차원 통계의 핵심 — <code>axis=0</code> 은 실무 최빈출

세 개 모두 **부록 C에 보충답안**을 실었다. 특히 실습 5의 `mask.sum() / mask.mean()`과 실습 6의 `axis=0`은 오늘 배운 것 중 실무 비중이 가장 큰 부분이라 반드시 손으로 쳐 보길 권한다.

③ 실습 9 — unpack=True로 shape이 뒤집혔다

Practice/10_02_example.py 실습 9

```
# x 제출 코드 — unpack=True 인데 변수 하나로 받음
rpm_and_torque = np.loadtxt(..., usecols=(4, 5), unpack=True)
print(rpm_and_torque.shape)      # (2, 40) — 교안 예상은 (40, 2)
get_rpm = rpm_and_torque[0]

# ✓ 방법 1 — 교안대로 (40, 2)
data = np.loadtxt(..., usecols=(4, 5))
print(data.shape)                # (40, 2)
rpm = data[:, 0]

# ✓ 방법 2 — unpack 을 쓸 거면 두 변수로
rpm, torque = np.loadtxt(..., usecols=(4, 5), unpack=True)

▶ (2, 40)
▶ (40, 2)
```

주의

결과값(이상 시점 1건, 평균 58.0)은 **맞게 나온다**. 하지만 교안이 요구한 shape 확인 결과가 (40, 2)가 아니라 (2, 40)으로 나오므로, 제출물로는 감점 요인이다. `unpack=True`는 **두 변수로 받을 때만** 쓰는 것이 원래 용도다.

④ 넘파이답지 않은 표현 — len() 대신 .size

Practice/10_02_example.py 실습 8

```
# x 리스트 사고방식
print(f"기준값 초과 개수: {len(torque_over_5)}")

# ✓ 배열 사고방식
print(f"기준값 초과 개수: {torque_over_5.size}")

# 1차원이면 둘 다 같은 값이지만, 2차원에서 갈린다
d = np.array([[1, 2, 3], [4, 5, 6]])
print("len :", len(d), " ← 행 개수만")
print("size:", d.size, " ← 전체 칸 수")

▶ len : 2 ← 행 개수만
▶ size: 6 ← 전체 칸 수
```

⑤ 반복되는 import와 중복 코드

✗ 원문	✓ 바로잡기	이유
파일 중간에 <code>import os · import numpy</code> <code>as np</code> 재선언	맨 위에 한 번만	PEP 8 — import는 파일 최상단
<code>np.set_printoptions(suppress=True)</code> 두 번 호출	맨 위에 한 번만	전역 설정이라 한 번이면 충분
경로를 <code>os.path.join</code> 과 문자열로 섞어 씀	한 방식으로 통일 — <code>os.path.join</code>	OS 이식성
<code>rpm</code> 변수를 실습마다 덮어씀	<code>rpm1 · rpm4</code> 처럼 구분	강사 코드도 이 방식을 씀

B-3. 교안 이탈자

교안 p48의 분산 예제 출력이 실제 계산과 다르다. 직접 검산했다.

교안 p48 — 174.0 이 아니라 170.0

```
unstable = np.array([60, 85, 65, 95, 70])
```

```
# 교안 p48 출력: 174.0
```

```
# 실제 값
```

```
print(round(unstable.var(), 1))
```

```
# 손으로 검산
```

```
mu = unstable.mean() # 75.0
```

```
print("편차 제곱:", (unstable - mu) ** 2) # [225 100 100 400 25]
```

```
print("합 850 / 5 =", 850 / 5)
```

```
▶ 170.0
```

```
▶ 편차 제곱: [225. 100. 100. 400. 25.]
```

```
▶ 합 850 / 5 = 170.0
```

개념

강사 배포 코드(10_02_11_statistics_39_260811_7.py)의 주석에는 # 170.0으로 올바르게 적혀 있다.

교안 슬라이드 쪽 숫자만 틀린 것이다. 이런 불일치를 만나면 코드를 실행한 결과가 정답이다. 슬라이드 숫자를 그대로 외우지 말고 직접 돌려 보는 습관이 이래서 필요하다.

실습 하이라이트

제출한 것 · 빠진 것 · 다음 시간

C-1. 오늘 나온 실습 전체

실습	교안 요구	제출 여부	한 줄 평
실습 3	정규화 — 0~1로 변환	미제출	브로드캐스팅 대표 실전 · 아래 보충
실습 4	이상 센서값 필터링	제출	다중 조건 잘 씀
실습 5	조건별 개수와 비율	미제출	불리언 sum/mean · 아래 보충
실습 6	axis로 센서별 통계	미제출	2차원 통계 핵심 · 아래 보충
실습 7	파일 데이터로 기초 통계	제출	loadtxt 인자 정확
실습 8	필터링과 통계 결합	제출	len() → .size 권장
실습 9	종합 분석	제출	unpack=True로 shape 뒤집힘
실습 1·2	(15일차 범위) 인덱싱·슬라이싱	제출	실습 2는 행·열 추출 누락

C-2. 미작성 실습 보충답안

실습 3 — 센서값 정규화하기

실습 3 보충답안

```
import numpy as np

# 회전수 측정 배열 준비
rpm = np.array([1551, 1408, 1498, 1433, 1425, 2861])
print(f"원본 : {rpm}")

# 최소값과 최대값 확인
rpm_min, rpm_max = rpm.min(), rpm.max()
print(f"최소·최대: {rpm_min} · {rpm_max}")

# 정규화 공식을 브로드캐스팅으로 적용
normalized = (rpm - rpm_min) / (rpm_max - rpm_min)
print(f"정규화 : {np.round(normalized, 2)}")

# 검증 — 최소값은 0, 최대값은 1
print(f"검산 : min {normalized.min()} · max {normalized.max()}")
```

```
▶ 원본 : [1551 1408 1498 1433 1425 2861]
▶ 최소·최대: 1408 · 2861
▶ 정규화 : [0.1  0.    0.06 0.02 0.01 1.   ]
▶ 검산 : min 0.0 · max 1.0
```


실습 5 — 조건별 개수와 비율 세기

실습 5 보충답안

```
import numpy as np
```

```
# 토크 배열 준비
```

```
torque = np.array([42.8, 46.3, 49.4, 4.6, 41.9, 65.7, 40.2, 60.7])
```

```
# 비교 조건으로 불리언 배열 생성
```

```
high = torque > 50
```

```
print(f"불리언 배열: {high}")
```

```
# 합(sum)으로 개수, 평균(mean)으로 비율
```

```
print(f"개수: {high.sum()}건")
```

```
print(f"비율: {round(high.mean(), 2)} ({high.mean() * 100:.1f}%)")
```

```
# 값까지 확인
```

```
print(f"초과 값: {torque[high]}")
```

▶ 불리언 배열: [False False False False False True False True]

▶ 개수: 2건

▶ 비율: 0.25 (25.0%)

▶ 초과 값: [65.7 60.7]

실습 6 — 센서별 기초 통계 구하기

실습 6 보충답안

```
import numpy as np
```

```
# 회전수·토크 이차원 배열 준비
```

```
data = np.array([
    [1551, 42.8],
    [1408, 46.3],
    [1498, 49.4],
    [2861, 4.6]
])
```

```
# axis=0 (열 방향) 으로 센서별 평균
```

```
mean_by_sensor = np.round(data.mean(axis=0), 2)
```

```
print(f"센서별 평균 : {mean_by_sensor}")
```

```
# 센서별 표준편차
```

```
std_by_sensor = np.round(data.std(axis=0), 2)
```

```
print(f"센서별 표준편차: {std_by_sensor}")
```

```
# 읽기 좋게 풀어 쓰기
```

```
rpm_m, tq_m = mean_by_sensor
```

```
rpm_s, tq_s = std_by_sensor
```

```
print(f"회전수: 평균 {rpm_m} · 표준편차 {rpm_s}")
```

```
print(f"토크 : 평균 {tq_m} · 표준편차 {tq_s}")
```

```
▶ 센서별 평균 : [1829.5 35.78]
```

```
▶ 센서별 표준편차: [597.73 18.15]
```

```
▶ 회전수: 평균 1829.5 · 표준편차 597.73
```

```
▶ 토크 : 평균 35.78 · 표준편차 18.15
```

C-3. 넘파이 단원 전체 마무리 — 하나의 파일로

15일차와 16일차를 합치면 넘파이 기초가 끝난다. 아래는 두 회차 내용을 한 흐름으로 이어 붙인 코드다. 포트폴리오 레포에 이 파일 하나만 있어도 "넘파이 다뤄 봤다"는 증거가 된다.

```
넘파이 기초 종합 — 15·16일차 전체
```

```
import numpy as np
```

```
PATH = "data/10_mct_tool.csv"
```

```
RPM_COL, TORQUE_COL = 4, 5
```

```
LOW_RPM = 1000
```

```
# ① 불러오기
```

```
data = np.loadtxt(PATH, delimiter=",", skiprows=1,  
                  usecols=(RPM_COL, TORQUE_COL), encoding="utf-8")
```

```
# ② 구조 확인
```

```
print(f"[구조] shape {data.shape} · dtype {data.dtype} · size {data.size}")
```

```
# ③ 추출
```

```
rpm, torque = data[:, 0], data[:, 1]
```

```
# ④ 통계 요약 (센서별)
```

```
mu, sd = data.mean(axis=0), data.std(axis=0)
```

```
print(f"[통계] 회전수 평균 {mu[0]:.1f} · 표준편차 {sd[0]:.1f}")
```

```
print(f"[통계] 토크 평균 {mu[1]:.1f} · 표준편차 {sd[1]:.1f}")
```

```
print(f"[통계] 회전수 중앙값 {np.median(rpm):.1f}")
```

```
# ⑤ 필터링 — 규칙 기준과 3시그마 기준 둘 다
rule = rpm[rpm < LOW_RPM]
lo, hi = mu[0] - 3 * sd[0], mu[0] + 3 * sd[0]
sigma = rpm[(rpm < lo) | (rpm > hi)]

print(f"[규칙] {LOW_RPM} 미만: {rule.size}건 {rule}")
print(f"[3시그마] 정상 {lo:.0f}~{hi:.0f} → 이상 {sigma.size}건 {sigma}")
print(f"[비율] 이상 시점 {rule.size / rpm.size * 100:.1f}%")
```

- ▶ [구조] shape (40, 2) · dtype float64 · size 80
- ▶ [통계] 회전수 평균 4212.6 · 표준편차 1144.9
- ▶ [통계] 토크 평균 10.1 · 표준편차 32.4
- ▶ [통계] 회전수 중앙값 4603.0
- ▶ [규칙] 1000 미만: 1건 [58.]
- ▶ [3시그마] 정상 778~7647 → 이상 1건 [58.]
- ▶ [비율] 이상 시점 2.5%

설비 적용

평균 4212.6인데 **중앙값은 4603**이다. 평균이 중앙값보다 390이나 낮다 — STEP 6에서 배운 **건강 점검 신호**다. 낮은 쪽에 이상치(58, 1216, 1217, 1241)가 있다는 뜻이고, 실제로 그렇다. 통계 두 줄만 비교해도 데이터의 문제가 드러난다.

C-4. 다음 시간 예고 — Pandas

교안 10_02의 마지막 문장이 이렇게 끝난다 — **"NumPy 기초 단단해짐 → 다음은 Pandas."** 강사도 수업 내내 "판다스로 넘어가면"이라는 말을 반복했다. 넘파이를 알게 뛴 이유가 여기 있다. **판다스가 쓰는 배경 기술로서만** 필요한 만큼 배운 것이다.

오늘 넘파이로 한 것	Pandas에서는	달라지는 점
data[:, 0] 번호로 열 추출	df["회전수"] 이름으로	열에 이름이 붙는다
data.mean(axis=0)	df.mean() · df.describe()	axis 신경 덜 씀
rpm[rpm < 1000]	df[df["회전수"] < 1000]	조건 필터링은 거의 동일
np.loadtxt(...)	pd.read_csv(...)	문자열·결측치도 처리
숫자만 담을 수 있음	숫자·글자·날짜 혼합 가능	실제 데이터에 맞는 구조
전부 같은 자료형	열마다 다른 자료형	표 데이터의 본모습

연결

걱정할 필요는 없다. 오늘 배운 조건 필터링·통계 개념은 Pandas에서 거의 그대로 쓰인다. `df[df["회전수"] < 1000]`은 오늘 배운 `rpm[rpm < 1000]`과 문법이 같다. 달라지는 건 "배열"이 "표"가 되고, 열을 번호 대신 이름으로 부른다는 것뿐이다. 오늘 것이 잡혀 있으면 다음 주는 오히려 쉽다.

C-5. 미리 보는 Pandas — 오늘 한 분석을 그대로

부록 C-3의 종합 분석을 Pandas로 옮기면 아래처럼 된다. 아직 배우지 않은 문법이니 얼마나 짧아지는지만 눈으로 확인하자.

다음 주에 배울 것 — 미리 보기

```
import pandas as pd

df = pd.read_csv("data/10_mct_tool.csv")

# 구조 확인 — shape·dtype·통계를 한 번에
print(df.shape)
print(df.describe())

# 열 추출 — 번호가 아니라 이름으로
rpm = df["회전수"]

# 필터링 — 문법이 넘파이와 거의 같다
print(df[df["회전수"] < 1000])
```

달라지는 점	넘파이 (오늘)	Pandas (다음 주)
헤더 처리	<code>skiprows=1</code> 로 버림	자동 인식 — 열 이름이 됨
열 지정	<code>usecols=4</code> 번호	<code>df["회전수"]</code> 이름
통계 한 번에	각각 따로 호출	<code>df.describe()</code> 하나로

연결

그렇다고 오늘 배운 게 헛수고는 아니다. Pandas의 내부가 전부 넘파이 배열이다. `df["회전수"].values`를 찍으면 오늘 다룬 ndarray가 그대로 나온다. 그리고 `df[df["회전수"] < 1000]`이라는 조건 필터링 문법은 오늘 배운 Boolean indexing과 완전히 같은 원리다. 오늘 것을 이해했으면 다음 주는 문법 몇 개만 더 없으면 된다.

오늘 넘은 선 — 데이터를 읽는 사람이 되기

어제까지는 배열을 **만들고 모양을 다루는** 법을 배웠다. 오늘은 그 배열에서 **의미를 꺼내는** 법을 배웠다. `rpm[rpm < 1000]` 한 줄로 40개 시점 중 고장 순간 하나를 찾아냈고, `mean`과 `std` 두 숫자로 "정상"이라는 것을 정의했다.

교안 p36의 문장이 오늘의 요약이다 — "**경험상 위험하다**"를 "**데이터상 몇 %가 기준을 넘었다**"로 바꾸는 것. 이것이 진짜 분석의 시작이고, 데이터 직군이 현장에서 하는 일의 본질이다.

오늘 할 수 있게 된 것	확인 질문	못 하면 돌아갈 곳
반복문 없이 배열 전체를 계산한다	브로드캐스팅이 성립하는 조건은?	STEP 1
값을 0~1 범위로 정규화한다	최댓값이 이상치면 어떤 문제가 생기나?	STEP 2
조건을 불리언 배열로 만든다	<code>v > 85</code> 의 결과 크기는 원본과 같은가?	STEP 3
조건에 맞는 값을 골라낸다	<code>v[v>85]</code> 와 <code>np.where</code> 의 차이는?	STEP 4
여러 조건을 묶는다	<code>and</code> 를 쓰면 왜 오류가 나나?	STEP 5
평균의 함정을 안다	평균과 중앙값이 크게 다르면 무슨 뜻인가?	STEP 6
정상 범위를 숫자로 정의한다	3-시그마 규칙을 한 줄로 쓸 수 있나?	STEP 7
2차원에서 센서별 통계를 낸다	센서별 통계는 <code>axis 0</code> 인가 <code>1</code> 인가?	STEP 8
CSV를 읽어 분석까지 잇는다	<code>skiprows</code> 를 빼면 무슨 오류가 나나?	STEP 9

복습 루틴 — 40분

오늘 내용은 15일차보다 **개념 비중이 높다**. 문법은 몇 개 안 되는데(&, |, `axis`, `loadtxt`) 그 뒤의 사고방식이 두껍다. 손으로 치는 것과 소리 내어 설명하는 것을 반반 섞자.



단계	무엇을	왜
① 부록 A 훑기 (5분)	치트시트의 네 표를 눈으로 넘긴다	오늘 나온 이름 전체를 다시 배치
② 보충답안 치기 (15분)	C-2의 실습 3·5·6을 보지 않고 쳐 본다	미제출 세 개가 실무 비중이 가장 크다
③ 종합 코드 이어 붙이기 (10분)	C-3을 처음부터 끝까지 실행해 본다	다섯 단계 흐름을 한 번에 체감
④ 부록 B 소리내어 (10분)	$x \rightarrow \checkmark$ 를 입으로 말하며 넘긴다	괄호 누락은 반드시 한 번은 겪는다

설비 적용

취업 관점에서 오늘 내용의 값어치는 15일차보다 크다. **제조 DX·AX 면접에서 "이상 탐지 해 본 적 있나요?"**는 단골 질문이고, 답할 거리가 오늘 다 나왔다 — 조건 필터링으로 이상 시점을 뽑고, 평균·표준편차로 정상 범위를 정의하고, 3-시그마로 자동 판정한다. 부록 A-5의 `find_anomalies` 함수를 GitHub 레포에 올려 두면 그 자체가 포트폴리오 항목이 된다.

다음 시간부터는 Pandas다. 배열에 **이름이 붙고**, 숫자가 아닌 값도 담기고, 표가 진짜 표처럼 다뤄진다. 넘파이는 오늘로 마무리되지만 사라지지 않는다 — **Pandas 안에서 계속 돌아간다.**

— 끝 —